



PH1A Series

PCI Express RC Connect EP Example Design

上海安路信息科技股份有限公司

APUG039 (v1.3) 2023 年 2 月

Confidential



目 录

目 录	I
1 概要	1
1.1 文件和目录结构	1
2 参考设计概述	3
2.1 验证环境	3
2.2 功能描述	3
2.2.1 系统框图	3
2.2.2 验证范围	5
3 RC/EP 功能概述	7
3.1 RC 端	8
3.1.1 UBUS 格式	8
3.1.2 UBUS 寄存器	9
3.1.3 IP GUI	12
3.2 EP 端	18
3.2.1 EP BAR 配置	19
3.2.2 IP GUI	19
3.3 中断实现方法	27
3.3.1 Legacy Int	27
3.3.2 MSI	28
3.3.3 MSIX	28
3.4 RC/EP 中断修改	29
3.5 顶层信号	29
3.6 自恢复模块	32



3.6.1 自恢复原理	32
3.6.2 IP 及参数配置	33
3.6.3 DBI 总线配置模块	34
4 仿真	35
4.1 运行仿真	35
4.2 仿真结构	35
5 编译与调试	39
5.1 配置说明	39
5.2 RC 连接 EP 测试过程	39
5.2.1 TLP 测试过程	42
5.2.2 中断测试过程	43
5.2.3 动态切换	44
5.3 EP 连接 PC 测试过程	46
5.3.1 TLP 测试过程	46
5.3.2 中断测试过程	49
6 参考文献	51
7 版本信息	52
免责声明	53



1 概要

本文档提供 SALPHOENIX®1A（以下简称 PH1A）系列 FPGA 的 PCIe RC 对接 EP 的参考设计，以便于用户快速掌握 PH1A 系列器件 PCIe 功能模块的使用。

在本参考设计中，两块 PH1A 系列 Demo 板分别配置为 RC 模式和 EP 模式。其中的 RC 向 EP 发送 TLP Request，EP 在接收到 RC 发来的 TLP 报文后由用户逻辑进行处理，对于 Posted 报文，将有效数据载荷存到 RAM 中；对于 Non-Posted 报文，用户逻辑发送 CPLD 包，RC 收到 CPLD 包并进行数据校验。RC 先将数据写入 EP，再从写入的地址读回数据，进行对比和校验，如果数据校验成功，证明该系统工作正常。

本文档提及到 PH1A 系列 FPGA，包括 PH1A90, PH1A180, PH1A400, PH1A60 不包含 PCIe IP，不在本文档讨论范围。

1.1 文件和目录结构

参考设计包含了所有需要的文件，包括 TD 工程，Linux 驱动等，参考设计文件结构如下：

表 1-1 参考设计结构表

目录			说明				
bit			可用于直接测试烧录的 .bit 文件				
doc			参考设计相关的文档				
driver			PCIe 驱动				
	Legacy int		测试 TLP 读写，Legacy int 中断驱动				
	MSI		测试 TLP 读写，MSI 中断驱动				
	MSIX		测试 TLP 读写，MSIX 中断驱动				
design			软件设计工程				
	90K		90K 器件参考设计例程				
		gen1_gen2		gen1_gen2 参考设计例程			
			impl		RC 和 EP 的软件工程		
				case_ep		PCIe EP 工程	
				case_rc		PCIe RC 工程	
			rtl		RC 和 EP 的源文件		
				case_ep		PCIe EP 源文件	



目录				说明	
			case_rc	PCIe RC 源文件	
			sim		工程仿真脚本及其波形文件
				modelsim	Modelsim 仿真文件
			tb		仿真 testbench 文件
		gen3		gen3 参考设计例程，目录结构与 gen1_gen2 一致	
	400K			400K 器件参考设计例程，目录结构与 90K 一致	



2 参考设计概述

使用本文档对应的参考设计，能够在安路提供的 PH1A 系列 Demo 板上完成 PCIe 硬核的简单验证。对于 90K 器件，使用 APC103_V1.0 DEMO 板，对于 400K 器件，使用 AP103_V2.0 DEMO 板，其中关于 400K 硬件单板的详细介绍，可参考《UG916_安路科技 AP103 开发板使用说明》。

参考设计提供了 gen1_gen2 和 gen3 两个参考例程，其中 gen1_gen2 默认提供 gen2x4 的示例，支持 TLP 测试和中断测试。gen3 默认提供 gen3x4 的示例，支持 TLP 测试和中断测试，但不建议进行速度位宽切换，所有参考例程都包含 EP 和 RC 两个工程，其中 RC 工程只能用于连接 EP 工程，而 EP 既可连接 RC，也可连接 PC 电脑进行测试。EP 的速度和位宽，可以通过修改寄存器配置文件实现，RC 可以通过串口访问寄存器修改。参考设计中 EP 和 RC 均采用 IP GUI 的方式生成，若用户希望修改 PCIe 的速度和带宽等配置，可自行通过 TD 软件，在 GUI 界面下修改配置。

同时，参考设计提供了完整的仿真设计，支持 modelsim 仿真环境。

2.1 验证环境

参考设计使用软硬件版本如下：

表 2-1 参考设计软件环境

Name	Version
FPGA ¹	PH1A90SBG484、PH1A400SFG900
TD Software	软件 5.6.2 Release
Modelsim	Modelsim SE-64 2020.4
OS	ubuntu22.04.1

注 1：PH1A 系列其他器件型号需对应具体原理图修改器件管脚约束

2.2 功能描述

2.2.1 系统框图

参考设计系统框图如下：

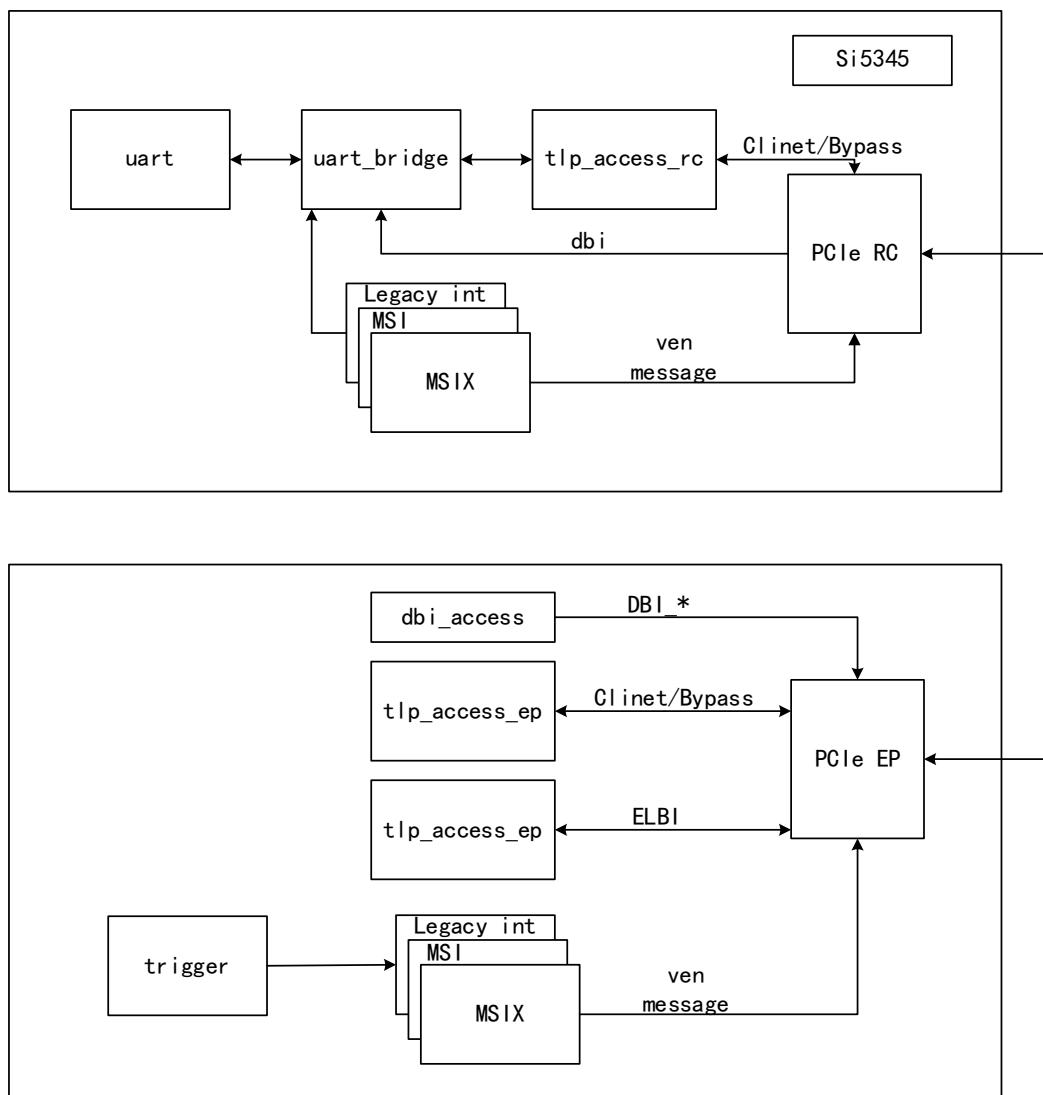


图 2-1 参考设计系统框图

PCIe RC 工程通过 UART，把 CFGWR0/CFGRD0/IOWR/IORD/MEMWR/MEMRD 这些 request，经由 UART 发出，经过 UBUS bridge，进入 tlp_access_rc。tlp_access_rc 把这些 request 组包，经 pcie_rc_core 的 clinet0_* 端口发出。pcie_ep_core 收到对端的 request，CFGWR0/CFGRD0 直接被 pcie_ep_core 接收，BAR0 的 request 通过 ELBI 总线送给 elbi_bar，BAR2 的 request 通过 trgt1_* 端口送给 tlp_access_ep。CFGWR0/CFGRD0 request，pcie_ep_core 会直接给出 completion。BAR0/BAR1 的读写，经过 ELBI 总线；其他 BAR 读写，经过 trgt1 接口。如果是对 BAR0 的 MEMWR/MEMRD，elbi_bar 执行读写操作，并返回响应。如果是对 BAR2 的 MEMWR/MEMRD 或 BAR4 的 IOWR/IORD，tlp_access_ep 收到 trgt1_* 端口的 request，执行读写操作，并对 MEMRD/IOWR/IORD 返回 completion。

PCIe EP 对 CFGWR0/CFGRD0/MEMWR/MEMRD/IOWR/IORD 返回 completion，经过 pcie_ep_core 和 PCIe 链接，给到 PCIe RC。pcie_rc_core 会把收到的 completion 经过 bypass_* 端口送到 tlp_access_rc。tlp_access_rc 把 completion 传递给 UBUS_bridge。

注：*elbi_bar 一次只能执行 1 个 Dword 读写。



*参考设计 EP 并不包含 DMA 控制器，也不能主动发起读写请求。

*本参考设计的 testbench 通过操作 UBUS 寄存器模拟了 UART 收发命令，与实际调试能够一一对应，并提供了 ubus 命令示例。

2.2.2 验证范围

参考设计涵盖了 PCIe TLP 层常用的报文，且支持三种中断 (Legacy int/MSI/MSIX) 的触发及检测。RC 端可以发起多种请求以获得 EP 端不同的信息，EP 端收到这些请求后，可以发出响应的完成报文。同时，RC 端支持配置 EP 端中断寄存器，使得 EP 端在收到中断触发的条件后，可以正常发起中断，而当 RC 收到这些中断时，也会做出相应的解析。RC 和 EP 端所支持的 TLP 报文如表 2-2 所示。

表 2-2 RC 和 EP 支持的报文

RC
Configuration Read Type 0 Request (CFGRD0)
Configuration Write Type 0 Request (CFGWR0)
I/O Read Request (IORD)
I/O Write Request (IOWR)
Memory Write Request (MEMWR)
Memory Read Request (MEMRD)
EP
Completion without Data (CPL)
Completion with Data (CPLD)
Legacy INTA
MSI Message
MSIX Message

测试过程中，RC 通过 UART 接收命令，并转成相应的 PCIe TLP 报文，与 EP 端进行交互，主要包含以下测试报文：

表 2-3 RC 和 EP 支持的报文

Type	Description	DataLen (byte)
cfg0wr	使能 EP 的 Memory 和 IO 空间	0
	设置 BAR0/2/4 的基地址	0



Type	Description	DataLen (byte)
cfg0rd	读取 EP 的 Device ID	0
	读取 BAR0/2/4 的基地址	0
Memwr	对 BAR0 写 1 个 DW	4
	对 BAR2 写 1 个 DW	4
	对 BAR2 写 2 个 DW	8
	对 BAR2 写 16 个 DW	64
	对 BAR2 写 32 个 DW	128
Memrd	读取 BAR0 读 1 个 DW	4
	读取 BAR2 读 1 个 DW	4
	读取 BAR2 读 2 个 DW	8
	读取 BAR2 读 16 个 DW	64
	读取 BAR2 读 32 个 DW	128
lowr	对 BAR4 写 1 个 DW	4
lord	读取 BAR4 读 1 个 DW	4
Legacy intx 中断	触发 EP Legacy intx 中断	0
MSI 中断	触发 EP MSI 中断	0
MSIX 中断	触发 EP MSIX 中断	0

注*：Legacy Int 中断，MSI 中断，MSIX 中断只有使能了相应的中断，才能进行触发。具体见 3.3 章节。



3 RC/EP 功能概述

EP 端的 `para_pcie_ep_case0.v` 和 RC 端 `para_pcie_rc_case0.v` 文件，包含了各种宏定义。通过宏定义设置各种参数，可由里面定义的参数，来选择使能哪一种接口或者使能哪一种中断。对于宏定义的选择，不建议用户修改接口定义，而只用来修改工程适配哪一种中断。RC/EP 主要参数定义如下表：

表 3-1 RC 和 EP 参数定义

Name	Function	RC/EP
DEVICE_TYPE	RC: “RC” , EP: “EP”	Both
XALIO_SIG_EN	PCIe Client0 接口使能	Both
XALI1_SIG_EN	PCIe Client1 接口使能	Both
TRGT1_SIG_EN	PCIe TRGT 接口使能	Both
BYPASS_SIG_EN	PCIe BYPASS 接口使能	Both
MSI_SIG_EN	PCIe MSI 接口使能	Both
MSIX_SIG_EN	PCIe MSI 接口使能	Both
LEGACY_SIG_EN	PCIe Legacy int 接口使能	Both
ELBI_SIG_EN	PCIe ELBI 接口使能	EP Only
DBI_SIG_EN	PCIe DBI 接口使能	Both
ATS_CAP_SIG_EN	PCIe ATS 接口使能	Both
TEST_SRIS	PCIe SRIS 功能测试，生成独立时钟	Both

当用户需要生成使能 Legacy Int 的参考设计，只需将 `LEGACY_INT_SIG_EN` 定义使能即可，当需要使能 MSI 参考设计，需要定义 `MSI_SIG_EN`，对于 MSIX，则需要使能 `MSI_MSIX_SIG_EN`。同理，RC 端和 PCIe IP 也需要使能相同的中断种类。才能适配整个中断链路，详见 3.4 章节。

3.1 RC 端

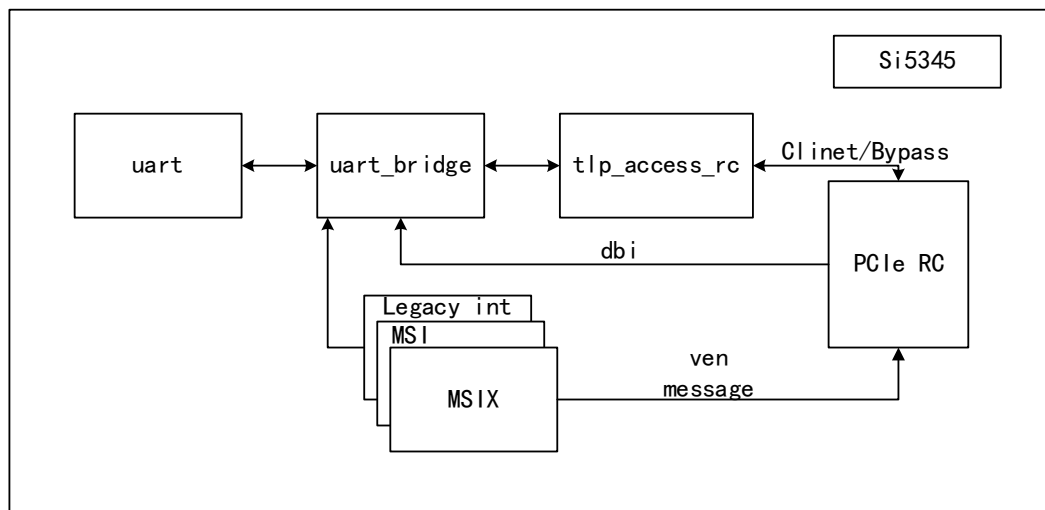


图 3-1 RC 端系统框图

首先 RC 端在上电后，内部 si5345 驱动模块，会驱动外部 si5345 晶振生成 100M 时钟，用于 RC 本身的参考时钟，并将该时钟通过金手指的连接送到 EP PCIe 参考时钟。

用户在 PC 上可以通过串口助手软件，发送控制字符到 UART 模块，经过 UART Bridge 之后，转换成对应的指令。UART Bridge 与 PCIe RC 和 tlp_access_rc 两个模块相连接。当用户需要修改 RC 端自身的寄存器配置，只需要通过 DBI 总线访问 PCIe RC 即可，访问对端 PCIe EP 设备，则需要通过 UART 控制 tlp_access_rc 模块，使其发送对应的存储器读写（MemWr/MemRd）或者配置请求（CfgWr/CfgRd）。

同时 RC 端支持三种中断的检测，但不支持同时存在，如果需要检测某一种中断，需要使能某一种中断，并在 RC 端调用相应的模块即可。检测中断是否收到，也是通过 UART 反馈到用户侧。

3.1.1 UBUS 格式

RC 通过 UART 与用户进行交互。UART 会将用户数据转换成 UBUS 的帧格式，UBUS 是基于 UART 的本地总线，与 RC 的 DBI 接口对接，可以借此完成配置寄存器的读写和发送读写请求，UBUS 总线时序图如下。

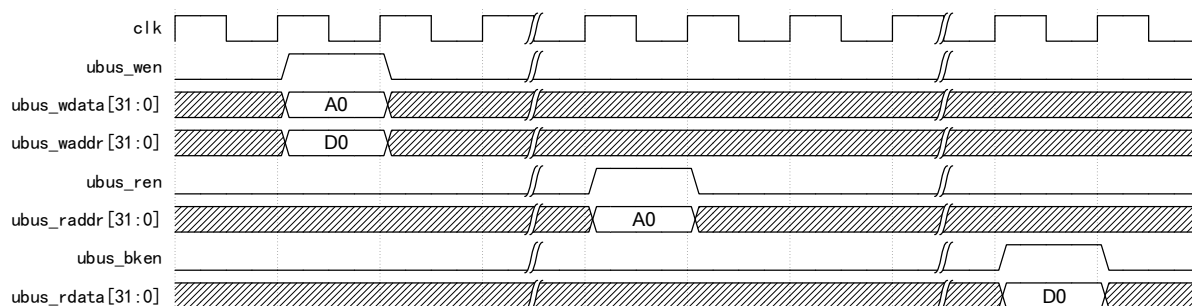


图 3-2 RC 端 ubus 总线时序

写方向,ubus_wen 使能信号为一拍,同时给出有效地址和数据,读方向,ubus_ren 使能信号为一拍,同时给出有效地址,随机数个周期后,bken 为读有效信号,拉高一拍并给出有效的 rdata。用户拉高 ubus_ren 读取数据后,必须等待 bken 信号为高后,才能进行下一步操作。

需要将 RC 端 UART 进行如下设置:

- 数据位 8
- 校验位为奇数校验
- 停止位 2
- 波特率: 115200

在 RC 端 Fabric 中,会把串口的数据帧转换成本地总线 UBUS,一次读写的帧格式是由 6 字节的帧头+4 字节地址+4 字节数据组成。其中帧头分为两类,即读和写。写数据帧如下:

41 42 43 44 45 46 00 00 00 04 12 34 56 78

其中 41 42 43 44 45 46 为帧头,00 00 00 04 为地址(高字节在前),12 34 45 78 为数据(高字节在前)读数据帧如下:

46 45 44 43 42 41 00 00 00 04 12 34 56 78

其中 46 45 44 43 42 41 为帧头,00 00 00 04 为地址(高字节在前),12 34 56 78 为填充数据,可任意给。当发送读数据帧时,RC 会以写数据帧的格式,将数据返回到串口。

3.1.2 UBUS 寄存器

参考设计中的自定义 UBUS 寄存器,其基地址 32' hFFFF_FF00 定义于 BDF.v 里。

表 3-2 UBUS 寄存器

地址	bits	属性	说明
32' hFFFF_FF00	[2] - dbi_ro_wr_dis	RW	DBI 寄存器里只读寄存器不能写的控制信号,1 不能写,



地址	bits	属性	说明
			0 可写; 此 bit 输出到 RC 的 drp_app_dbi_ro_wr_disable 输入 port
	[1] -- dbi_cs2		DBI 总线的 cs2。
32' hFFFF_FF01	[5] -- mw_en	RW	Memory write enable
	[4] - mr_en	RW	Memory read enable
	[3] -- iow_en	RW	IO write enable
	[2] -- ior_en	RW	IO read enable
	[1] - cfg0w_en	RW	Configuration write 0 enable
	[0] -- cfg0r_en	RW	Configuration read 0 enable
32' hFFFF_FF02	Null	WO	对此寄存器做写动作, 就会产生一次上面 6 个操作中被使能的那个 request。
32' hFFFF_FF02	[5] -- mw_op_over	RO	Memory write over
	[4] - mr_op_over		Memory read over
	[3] -- iow_op_over		IO write over
	[2] -- ior_op_over		IO read over
	[1] - cfg0w_op_over		Configuration write 0 over
	[0] -- cfg0r_op_over		Configuration read 0 over
32' hFFFF_FF03	[31:0]- mw_addr [31:0]	RW	Memory write 地址低 32 位
32' hFFFF_FF04	[31:0]-mw_addr [63:32]	RW	Memory write 地址高 32 位
32' hFFFF_FF05	[31:0]- mw_data [31:0]	RW	Memory write 数据
32' hFFFF_FF06	[31:0]-mw_data [63:32]	RW	
32' hFFFF_FF20	[31:0]-mw_data [95:64]	RW	
32' hFFFF_FF21	[31:0]-mw_data [127:96]	RW	



地址	bits	属性	说明
32' hFFFF_FF07	[11:0] - mw_len	RW	Memory write 数据长度, 单位 byte
	[23:16]-mw_data_be		Memory write 数据 byte enable [23:20] - first BE [19:16] - last BE
32' hFFFF_FF08	[31:0]-mr_addr[31:0]	RW	Memory read 地址低 32 位
32' hFFFF_FF09	[31:0]- mr_addr[63:32]	RW	Memory read 地址高 32 位
32' hFFFF_FF0A	[11:0] -- mr_len	RW	Memory read 数据长度, 单位 byte
	[23:16] -- mr_data_be		Memory read 数据 byte enable [23:20] - first BE [19:16] - last BE
32' hFFFF_FF0B	[31:0]—wr_data	RO	Memory read 数据
32' hFFFF_FF0C	[31:0] --iow_addr	RW	IO write 地址
32' hFFFF_FF0D	[31:0] --iow_data	RW	IO write 数据
32' hFFFF_FF0E	[31:0] --ior_addr	RW	IO read 地址
32' hFFFF_FF0F	[31:0] --ior_data	RO	IO read 数据
32' hFFFF_FF10	[31:0] -- cfg0w_addr	RW	Configuration write 0 地址
32' hFFFF_FF11	[31:0] -- cfg0w_data	RW	Configuration write 0 数据
32' hFFFF_FF12	[31:0] -- cfg0r_addr	RW	Configuration read 0 地址
32' hFFFF_FF13	[31:0] -- cfg0r_data	RO	Configuration read 0 数据
32' hFFFF_FF14	[3] - INTD [2] - INTC	RO	INTA/B/C/D 中断状态
	[1] - INTB		



地址	bits	属性	说明
	[0] -- INTA		
32' hFFFF_FF14	[3] - INTD clear [2] - INTC clear [1] - INTB clear [0] -- INTA clear	WO	分别针对 INTA/B/C/D 的清零信号

上表中的寄存器属性，如下表所示。

表 3-3 寄存器属性说明

寄存器属性	说明
RW	可读可写
RO	只读
WO	只写

在 doc 目录中有完整的串口命令格式 `uart_cmd_file.txt` 文件，供用户参考。

3.1.3 IP GUI

RC 端 PCIe 通过 IP GUI 方式生成，配置界面选择如下：

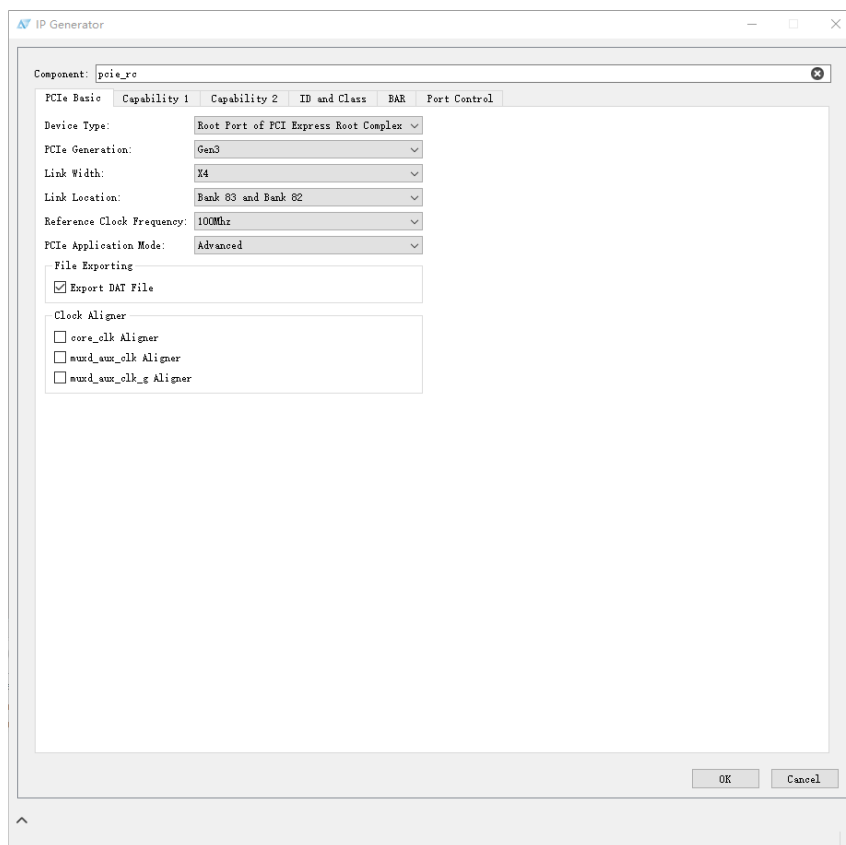


图 3-3 RC 端 PCIe GUI 1

目前参考设计支持 gen1/gen2/gen3，且位宽为 x1/x2/x4 的情况。注：PCIe Application Mode 需选择 Advanced 才能对 Capability1, Capability2 进行配置, Export DAT File 用于自恢复模块生成的 .dat 文件，可用于 rom 初始化，详见 3.6 章节。

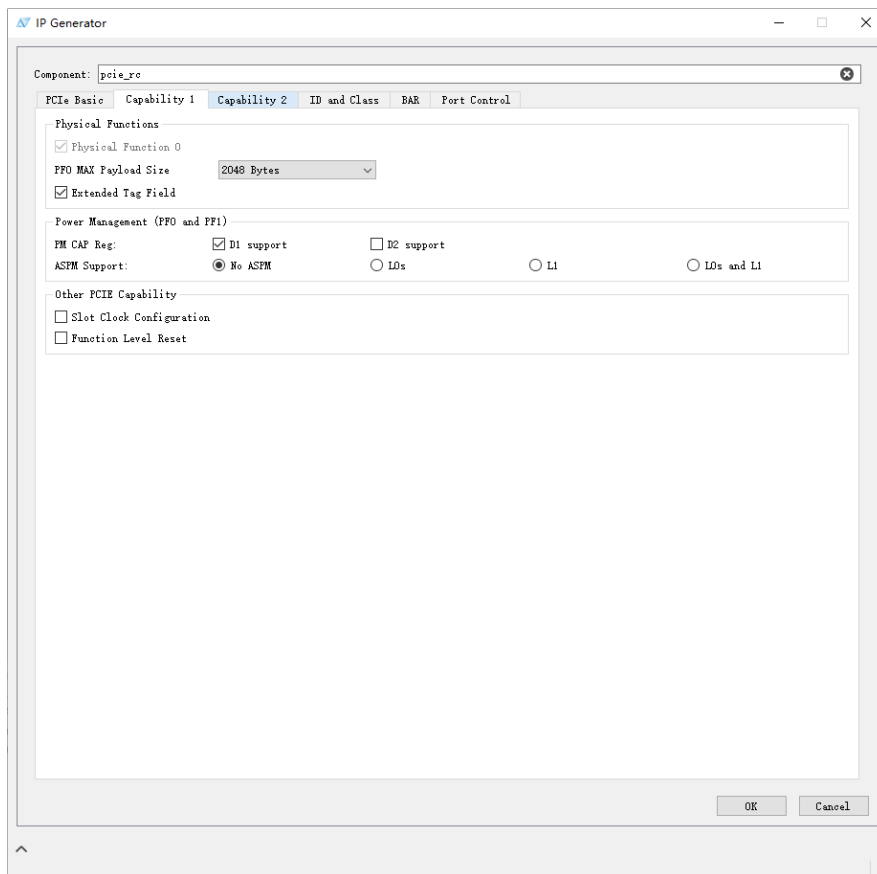


图 3-4 RC 端 PCIe GUI 2

保持默认配置即可。

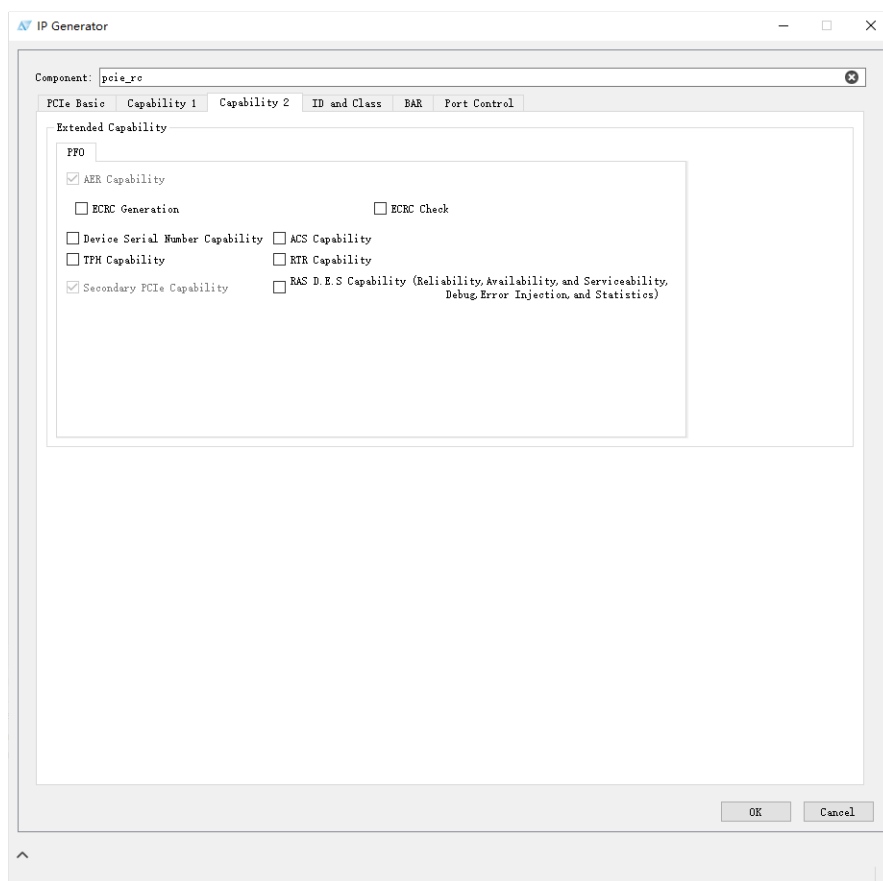


图 3-5 RC 端 PCIe GUI 3

保持默认配置即可。

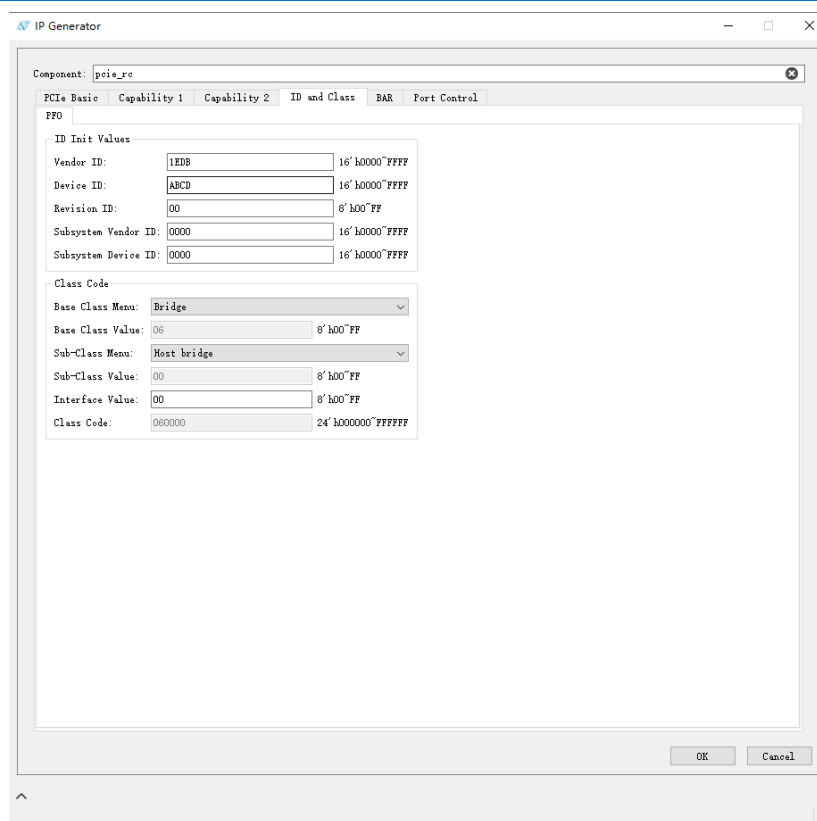


图 3-6 RC 端 PCIe GUI 4

保持默认配置即可。

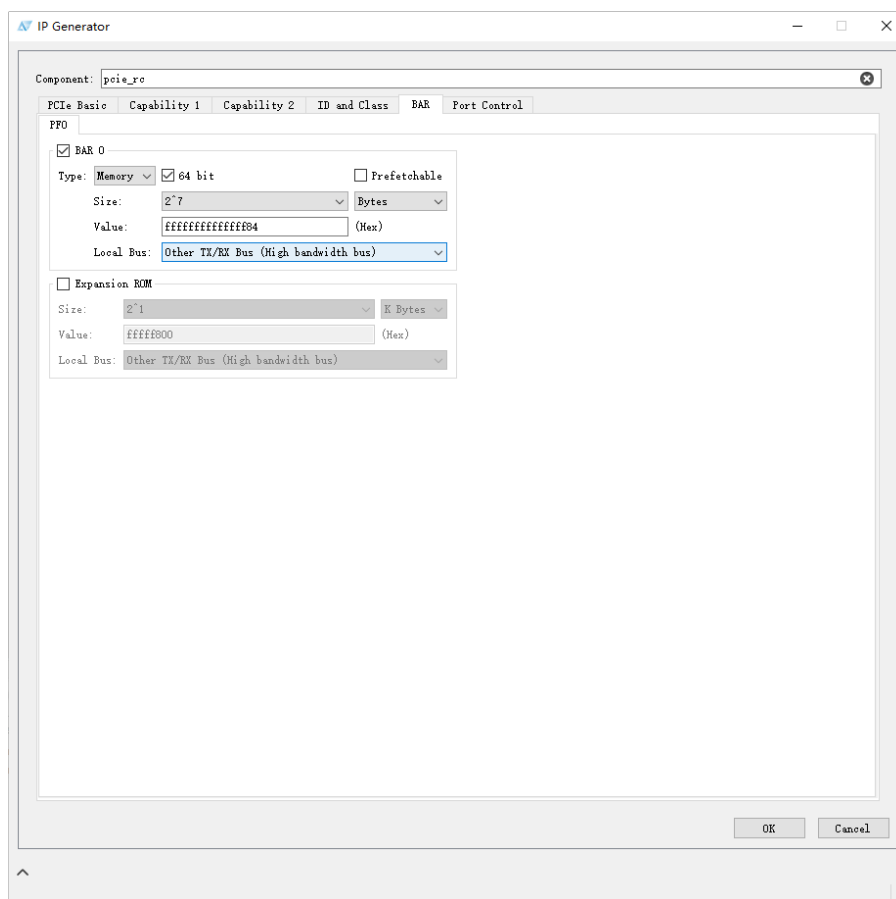


图 3-7 RC 端 PCIe GUI 5

保持默认配置即可。

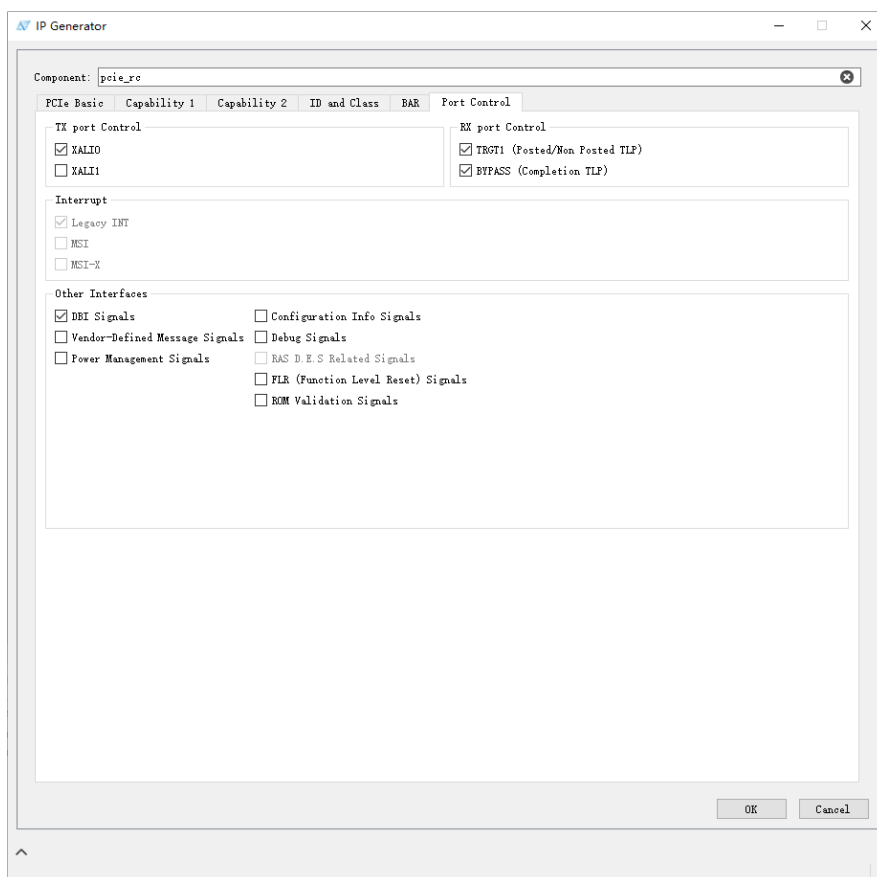


图 3-8 RC 端 PCIe GUI 6

RC 端接口使能，目前用到了 XALIO/TRGT1/BYPASS/DBI 接口，如果用户修改接口设置，需要在宏定义文件中，同样定义该接口。

3.2 EP 端

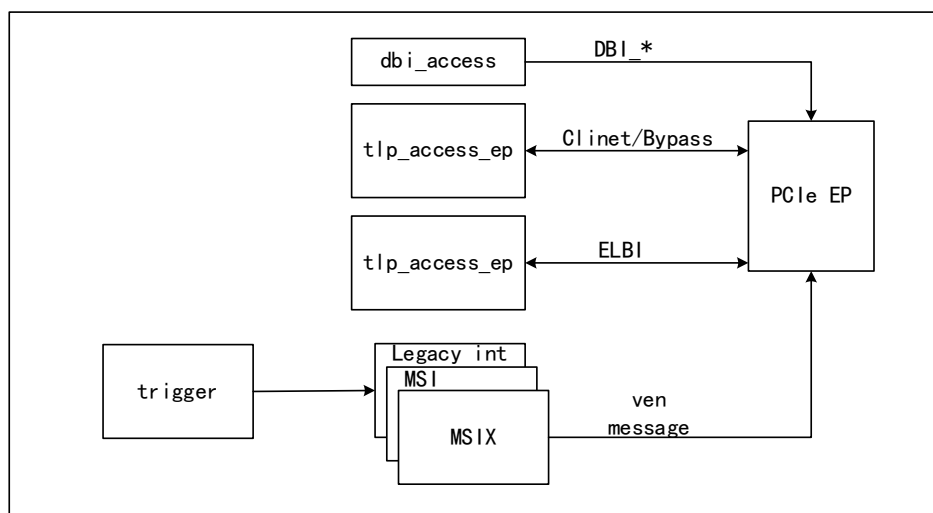


图 3-9 EP 端系统框图



EP 通过 TD 内部 IP GUI 的方式生成并调用。主要验证 EP 端 ELBI/Client/Target 接口的功能是否正常且能够正常传输数据。当读写 BAR0 时，会通过 ELBI 接口进行访问，当读写 BAR2/4 时，会通过 Client/Target。同时 EP 支持通过 DBI 接口，修改自身寄存器配置。

EP 支持三种中断的生成，即 Legacy INT/MSI/MSIX。三种中断在参考设计中只能同时使能一种，且需要 RC 和 EP 共同支持同一种中断。在 RC 和 EP 参考设计中，如果需要尝试不同的中断，需要修改 PCIe IP 并选择相应的中断同时修改参数定义。

3.2.1 EP BAR 配置

EP 里，有 3 个 BAR，具体设置如下表，也可以配置其他合理值。

表 3-4 EP BAR 配置表

BAR NUM	BAR0	BAR2	BAR4
Memory/IO	Memory	Memory	IO
Non-prefetchable/Prefetchable	Non-prefetchable	Non-prefetchable	/
32bit/64bit	32bit	32bit	/
Space size	1M Bytes	1M Bytes	16 Byte
Base address	32'h8000_0000	32'h4000_0000	32'h0000_4000

Base address 是建链成功后，RC 给 EP 配置的基地址。

EP 中 BAR 的实现是由 FPGA 内部 Fabric 中 RAM 完成的。BAR0 实际的物理存储是 1K Bytes。如果超过 1K Bytes 的第 2/3/4...个 1K，也会映射到物理的 1K Bytes。BAR2 实际的物理存储是 2K Bytes。如果超过 2K Bytes 的第 2/3/4...个 2K，也会映射到物理的 2K Bytes。BAR4 实际的物理存储只有 4Byte。访问 BAR4，地址将被忽略。

3.2.2 IP GUI

EP 端 PCIe 通过 IP GUI 方式生成，配置界面选择如下：

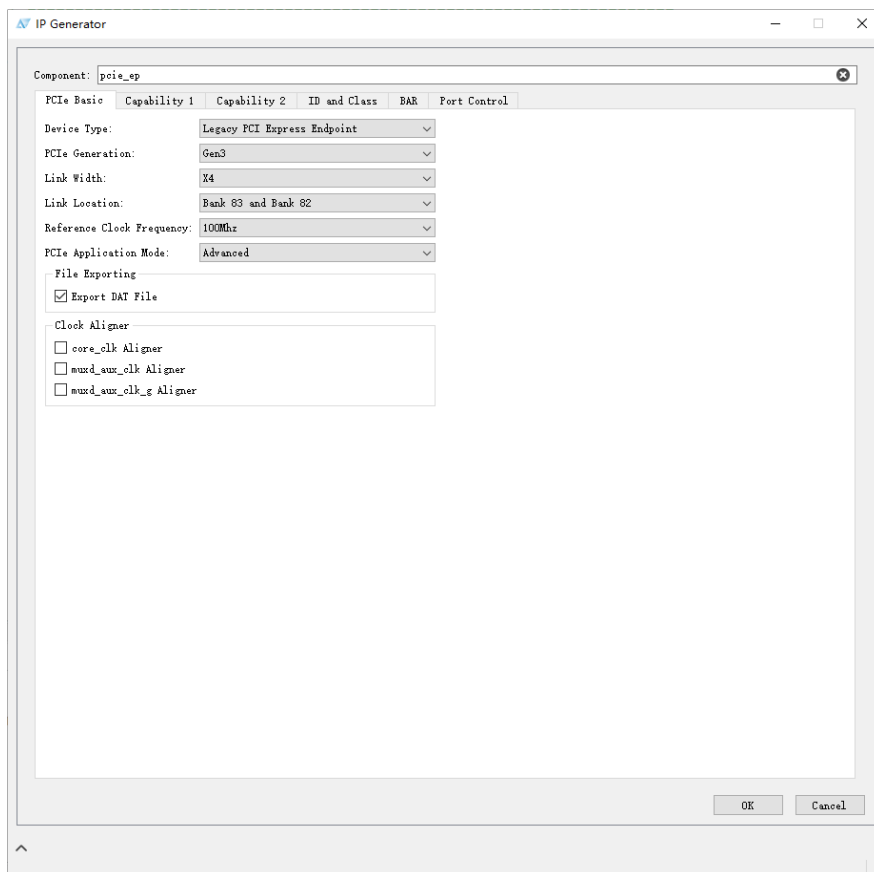


图 3-10 EP 端 PCIe GUI 1

目前参考设计支持 gen1/gen2/gen3, 且位宽为 x1/x2/x4 的情况。注: Device Type 选择类型为 Legacy PCI Express Endpoint 用于参考例程中对 IO 读写进行测试, PCIe Application Mode 需选择 Advanced 才能对 Capability1, Capability2 进行配置, Export DAT File 用于自恢复模块生成的 .dat 文件, 可用于 rom 初始化, 详见 3.6 章节。

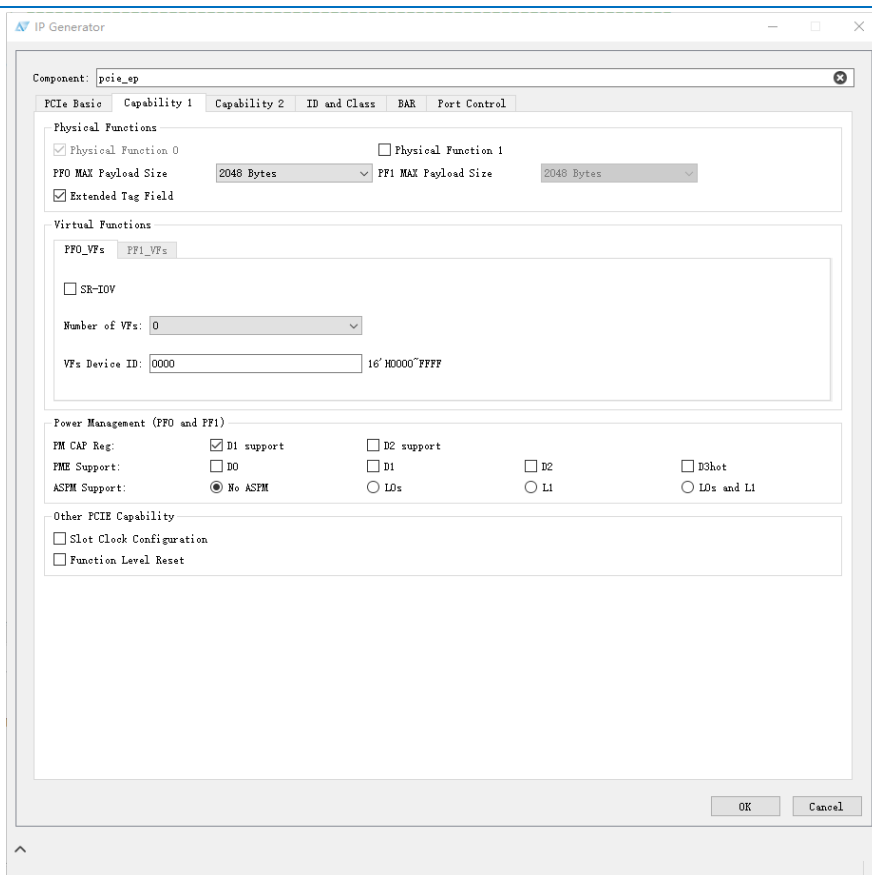


图 3-11 EP 端 PCIe GUI 2

保持默认配置即可。

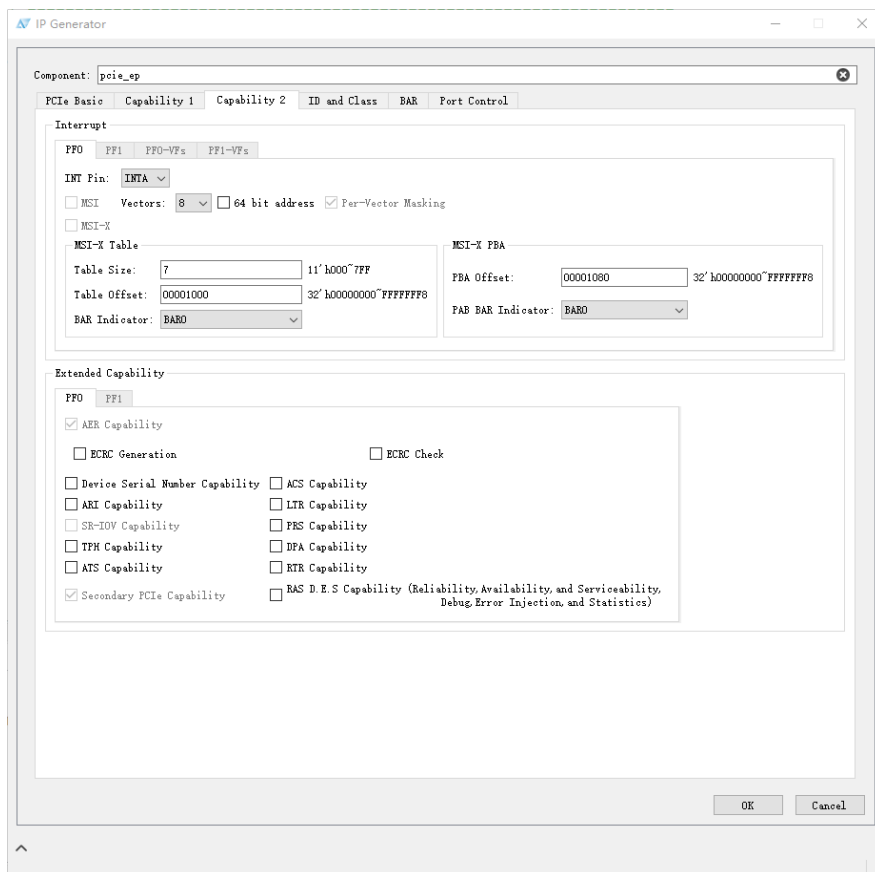


图 3-12 EP 端 PCIe GUI 3_1

使能 Legacy Int 时，Capability2 配置界面如上，使能 INTA 即可。

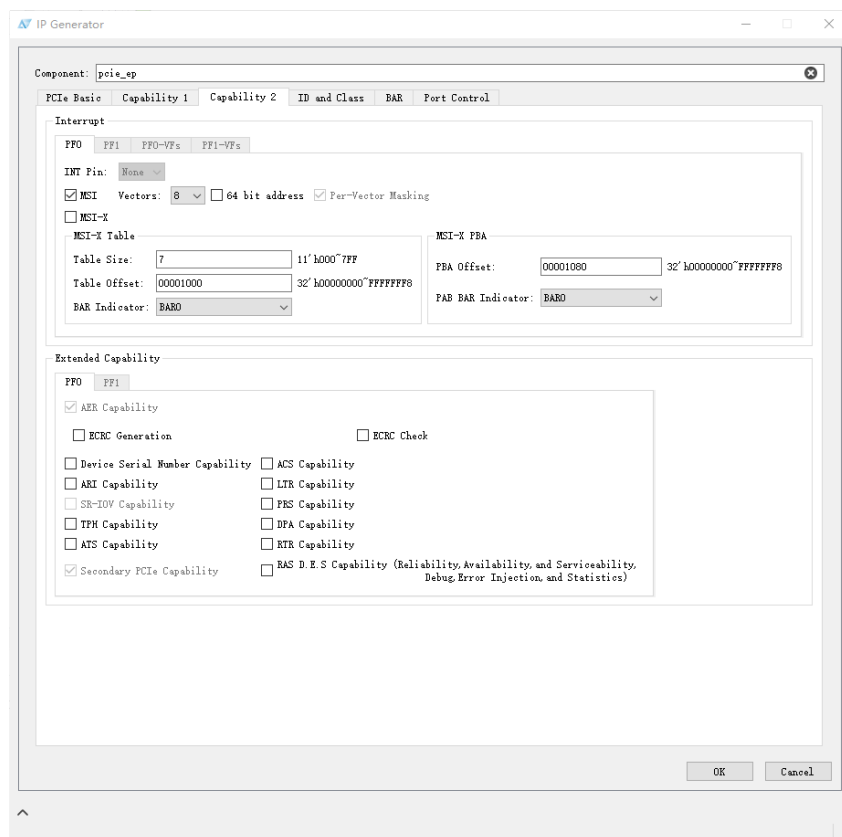


图 3-13 EP 端 PCIe GUI 3_2

使能 MSI 时，Capability2 配置界面如上，Vectors 选择 8 即可。

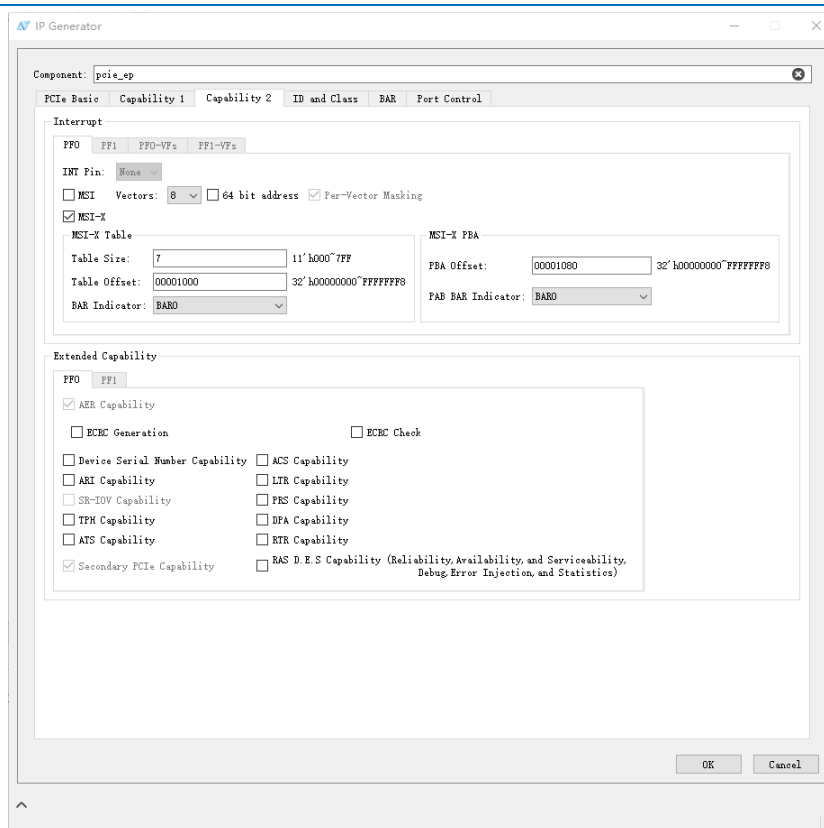


图 3-14 EP 端 PCIe GUI 3_3

使能 MSIX 时，Capability2 配置界面如上，最多使能 7 个中断向量，Table 和 PBA 都存放在 BAR0，TABLE offset 位置在 BAR0 的 0x1000，PBA offset 位置在 BAR0 的 0x1080，需要和代码配合一起定义。

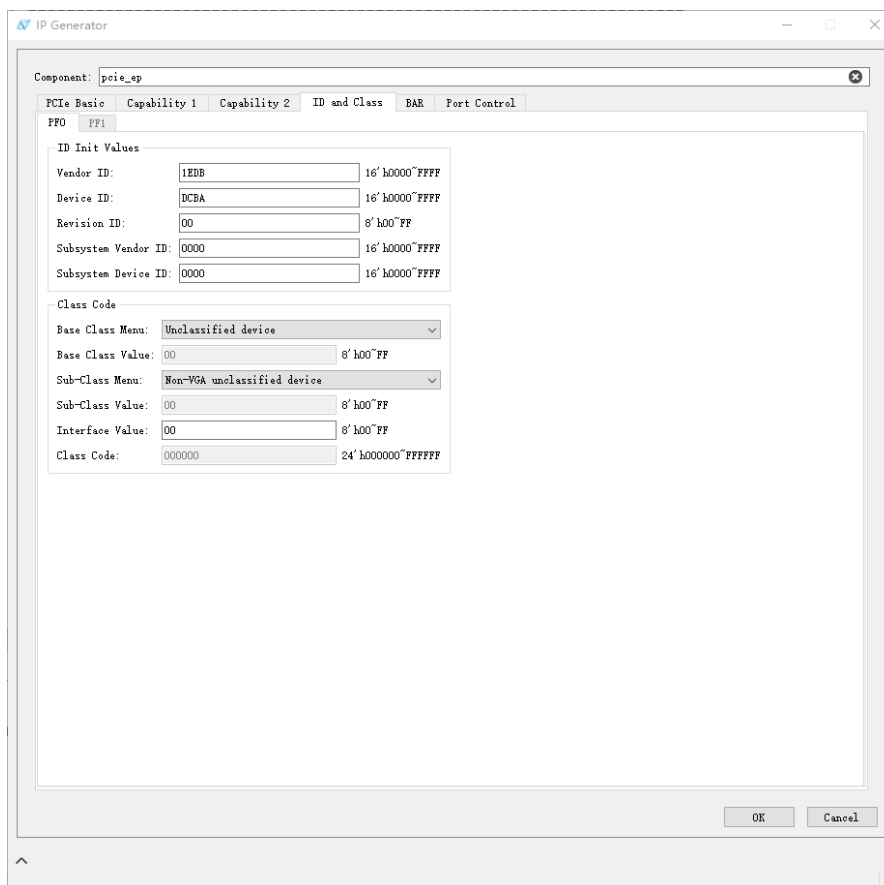


图 3-15 EP 端 PCIe GUI 4

保持默认配置即可。

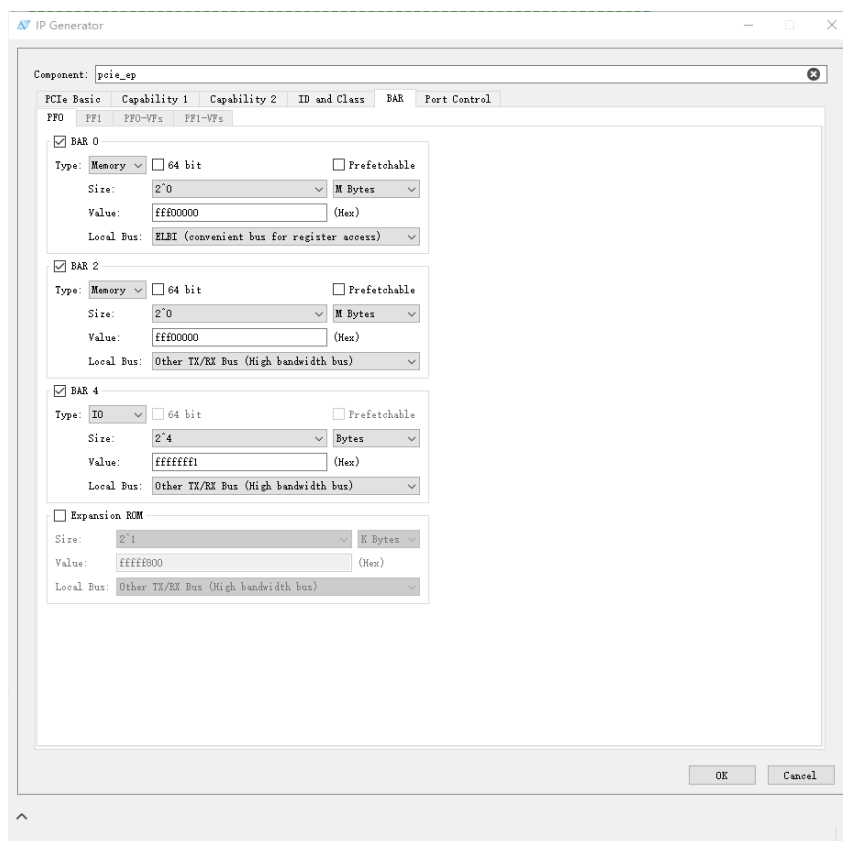


图 3-16 EP 端 PCIe GUI 5

各个 BAR 的大小在 3.2.1 章节已经定义，其中 BAR0 默认使能 ELBI 接口。

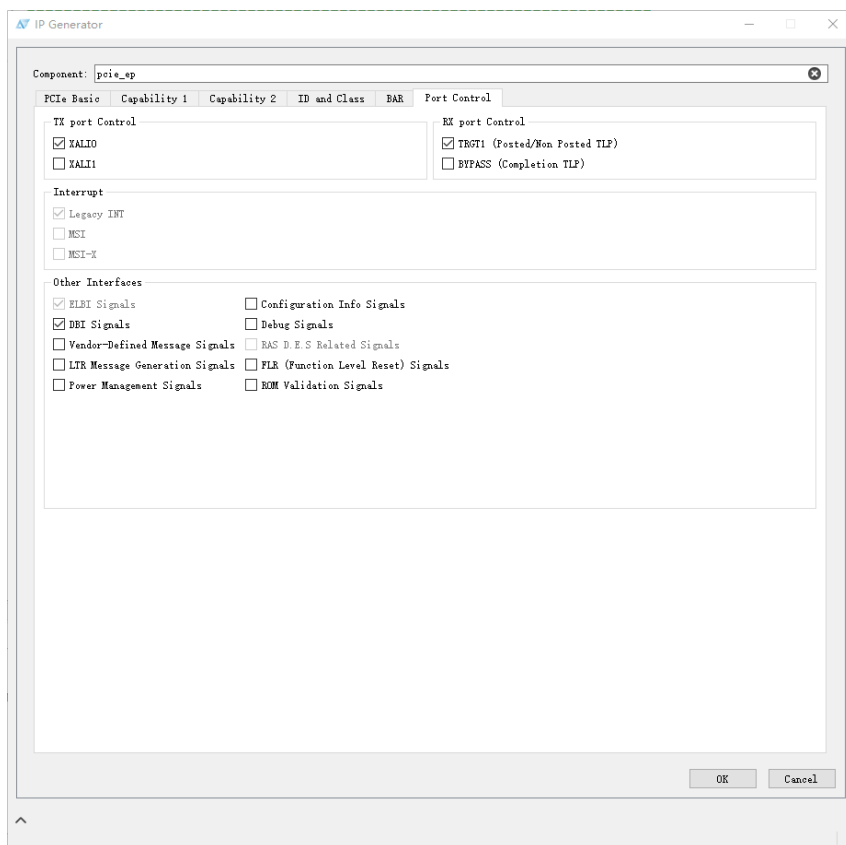


图 3-17 EP 端 PCIe GUI 6

EP 端接口使能，目前只用到了 ELBI/XALIO/TRGT1/DBI 接口，如果用户修改接口设置，需要在宏定义文件中，同样定义该接口。

3.3 中断实现方法

所有中断的中断源都是通过 EP 端 Demo 板卡上绑定的中断按键来触发的，当按下后，会经过 10ms 的消抖，然后根据定义不同的中断，进入到不同的模块，以生成中断。

3.3.1 Legacy Int

当检测到用户按键输入后，会将 `sys_int[0]` 拉低 (bit0 表示 PF0)，控制器检测到 `sys_int[0]` 从低到高的过程后，会发送一个 `Assert_INTx` 的 message 报文；控制器检测到 `sys_int[0]` 从高到低的过程后，会发送一个 `Deassert_INTx` 的 message 报文。当控制器发送完 `Assert_INTx` 的 message 后，输出持续一个时钟周期的 `assert_intx_grt` 高脉冲作为提示；当控制器发送完 `Deassert_INTx` 的 message 后，输出持续一个时钟周期的 `deassert_intx_grt` 高脉冲作为提示。控制器输出 `cfg_int_pin` 信号指示配置空间中 `Interrupt Pin` 寄存器的值。

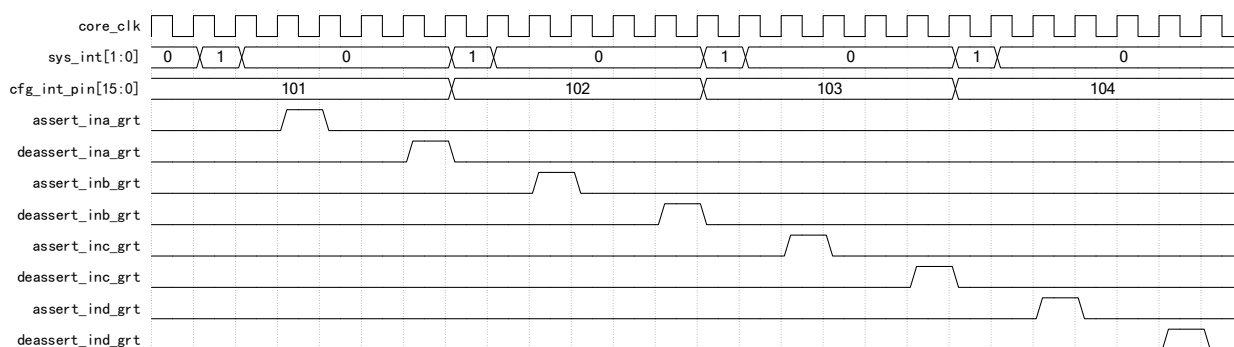


图 3-18 Legacy Int 生成

3.3.2 MSI

当检测到用户按键输入后, `msi_process` 模块通过控制 PCIe IP MSI 信号来生成 MSI 中断。通过改变 `message data` 的 `vector[4:0]` 信号, 参考设计最多提供 8 个 MSI 中断。下图描述了一个示例, 用户逻辑在 `cfg_msi_en` 拉高时, 请求控制器向上游发送 MSI 消息。`ven_msi_req` 保持拉高状态, 直到控制器拉高 `ven_msi_grant`。内部 MSI 仲裁器对来自不同功能的 MSI 请求执行仲裁。`ven_msi_grant` 是确认 `ven_msi_req` 的单周期脉冲。无需拉低 `ven_msi_req`, 然后再将其拉高。当 `ven_msi_req` 保持拉高时, 控制器将生成另一个 MSI。

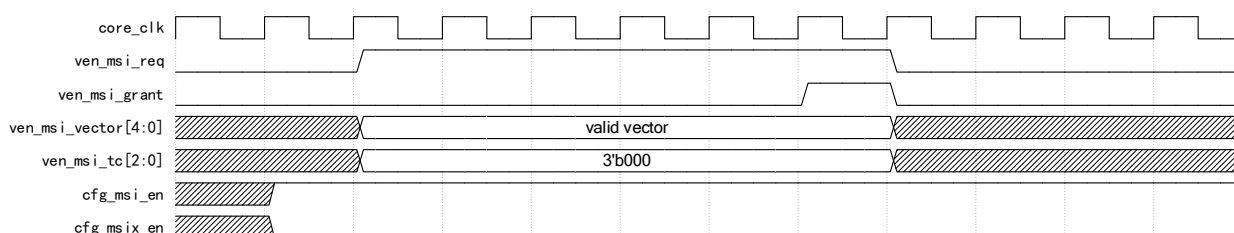


图 3-19 MSI 生成

3.3.3 MSIX

MSIX 是 MSI 的扩展和增强。MSI 有它自身的局限性, MSI 最多支持 32 个中断, 且要求中断向量连续, 而 MSIX 没有这个限制, 且支持的中断数量更多。此外, MSIX 的中断向量信息并不直接存储在 `capability` 中, 而是在一块特殊 Memory 中。由此, 在 `msix_process` 模块中, 例化了一块 32 位宽, 64 深度的 `ram`, 用于存放 EP 端配置的不连续的中断向量, 当用户按下按键时, `msix_process` 模块会依次发送 MSIX 向量 0 到 3, 一共 4 个, 以此往复。

下图显示了 MSIX 中断请求一个示例, 该示例在 `cfg_msix_en` 声明时请求控制器向上游发送 MSIX 消息。`ven_msi_req` 保持高电平状态, 直到控制器拉高 `ven_msi_grant`。

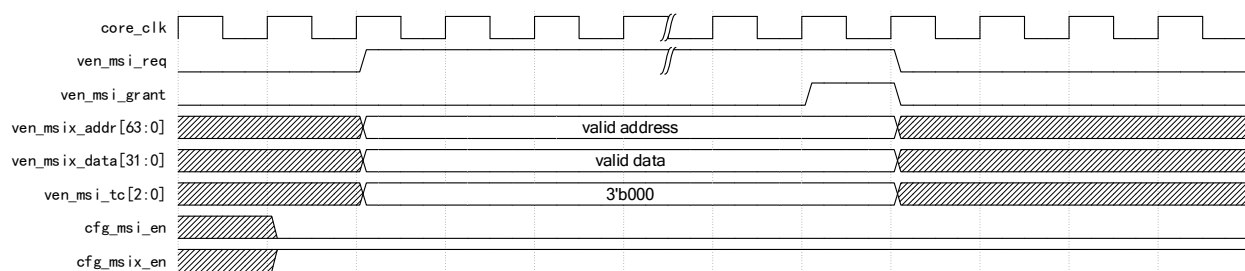


图 3-20 MSIX 生成

3.4 RC/EP 中断修改

参考设计支持三种中断的生成和校验，但是只能同时支持一种中断。默认生成为 Legacy int，如果需要测试 MSI 和 MSIX，则需要修改 IP 和参数。修改步骤如下：

1. EP 端 PCIe IP 为 GUI 生成，所以需要修改 PCIe IP，请按照上一节 3.2 中，EP 端 IP 的配置，选择对应中断该如何生成 IP，有详细的解释。
2. 修改 EP 端和 RC 端参数，在 EP 端和 RC 端各自存在 para_pcie_rc/ep_case0.v，用户需要修改里面的配置，以使能对应的中断。

例如，需要从 Legacy int 中断修改为 MSI 中断，则先需要修改 EP 端 PCIe IP，使其能够支持 MSI 中断，然后在修改 EP 和 RC 端定义，将：

```
`define LEGACY_INT_SIG_EN
```

注释，然后解注释：

```
`define MSI_SIG_EN
```

即可。同理，对 MSIX 中断同样适用，需要解注释：

```
`define MSIX_SIG_EN
```

即可，参考设计虽然支持三种中断，需要注意三种中断同时只能设置一种，且需要配合 EP 端 IP 才能生效。

3.5 顶层信号

对于 90K 器件例程 RC 工程顶层信号如下表所示：

表 3-5 RC 端顶层信号

名称	位置	描述
app_auxclk_i	N18	FPGA 输入时钟，50M
app_power_up_rst_n	R17	FPGA 输入复位



名称	位置	描述
pcie_clock_rst_n	H10	app_init_rst 复位
pcie_perest_o	F13	PCIe 金手指复位
rx_d	A11	Uart 接收管脚
tx_d	H13	Uart 发送管脚
G_led1	M18	PCIe support.v 模块 user_link_up 信号输出
G_led2	M17	coreclk 时钟分频输出
intrb_n	N17	RC 端 100M 参考时钟生成，配 置外部 si5345 晶振。
Rstb_n	M21	
Sda_io	L21	
OEB	M20	
Sck_o	L18	
SI5345_FINC	N22	
SI5345_FDEC	K17	
SI5345_IN_SEL1	N20	
SI5345_IN_SEL0	K21	

EP 工程顶层信号如下表所示：

表 3-6 EP 端顶层信号

名称	位置	描述
app_auxclk_i	N18	FPGA 输入时钟，50M
app_power_up_rst_n	R17	FPGA 输入复位
pcie_perest_i	F13	PCIe 金手指复位
local_int	H10	中断触发
R_led1	M18	PCIe support.v 模块 user_link_up 信号输出
R_led2	M17	coreclk 时钟分频输出



对于 400K 器件例程 RC 工程顶层信号如下表所示：

表 3-7 RC 端顶层信号

名称	位置	描述
app_auxclk_i	U27	FPGA 输入时钟，50M
app_power_up_rst_n	L13	FPGA 输入复位
pcie_perest_o	T25	PCIe 金手指复位
pcie_clock_rst_n	K13	app_init_rst 复位
rx_d	R24	Uart 接收管脚
tx_d	U30	Uart 发送管脚
G_led1	J11	auxclk 时钟分频输出
G_led2	K11	coreclk 时钟分频输出
G_led3	J13	PCIe IP rd1h_link_up 信号输出
G_led4	C11	输出低电平
R_led1	J14	PCIe support.v 模块 user_link_up 信号输出
R_led2	H11	输出低电平
R_led3	J12	RC 收到串口启动 TLP 传输使能信号，闪烁 1s 左右
R_led4	J17	RC 收到 EP 端对应 TLP 报文，闪烁 1s 左右
intrb_n	T28	RC 端 100M 参考时钟生成，配置外部 si5345 晶振。
Rstb_n	T26	RC 端 100M 参考时钟生成，配置外部 si5345 晶振。
Sda_io	R30	
OEB	R26	
Sck_o	R28	
SI5345_FINC	P28	



名称	位置	描述
SI5345_FDEC	T30	
SI5345_IN_SEL1	T27	
SI5345_IN_SEL0	U28	

EP 工程顶层信号如下表所示：

表 3-8 EP 端顶层信号

名称	位置	描述
app_auxclk_i	U27	FPGA 输入时钟，50M
app_power_up_rst_n	L13	FPGA 输入复位
pcie_perest_i	T25	PCIe 金手指复位
Local_int	K14	中断触发
G_led1	J11	auxclk 时钟分频输出
G_led2	K11	coreclk 时钟分频输出
G_led3	J13	PCIe IP rd1h_link_up 信号输出
G_led4	C11	输出低电平
R_led1	J14	PCIe support.v 模块 user_link_up 信号输出
R_led2	H11	输出低电平
R_led3	J12	输出低电平
R_led4	J17	输出低电平

3.6 自恢复模块

3.6.1 自恢复原理

PCIe IP 的配置信息是由 IP GUI 生成的.txt 文件来控制的，当 FPGA 上电后，会从 bit 流中自动加

载.txt 文件信息，如果复位 PCIE IP，会清空 PCIE 的配置信息，导致 PCIE IP 处于上电默认状态。

参考设计提供了一种 PCIE 自恢复方案，即在 PCIE IP 复位后，通过拉低 ltssm_enable 信号，延迟 PCIE 链路训练，并控制 PCIE DBI 接口，将配置信息重新写入到 PCIE 中，当配置信息写入完毕后，会释放 ltssm_enable 信号和 DBI 总线。

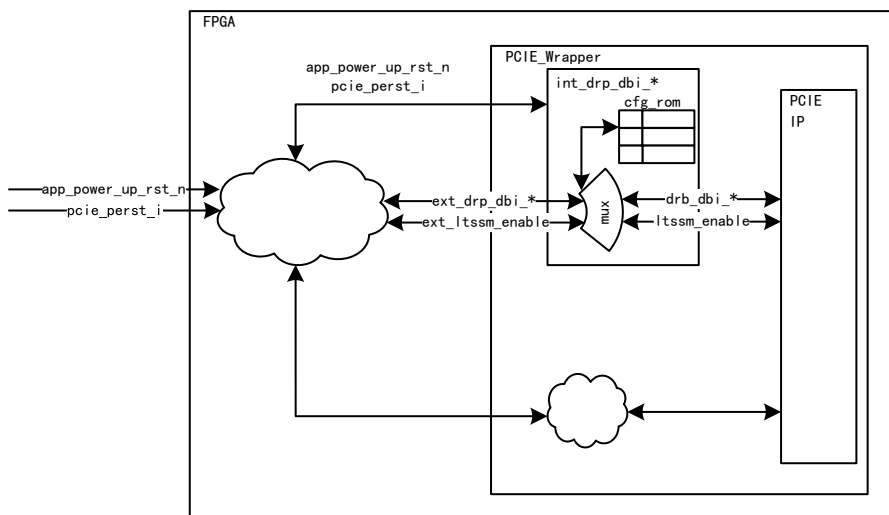


图 3-21 PCIE 自恢复模块

dbi_init_mux 模块中例化一个 ROM，深度 128，位宽 68，存储了 PCIE 的配置信息，即 PCIE GUI 生成的.dat 文件。复位之后，dbi_init_mux 模块会先于用户来控制 ltssm_enable 信号和 DBI 总线，只有当 init_dbi 写入完全部的配置信息后，才会释放 ltssm_enable 信号和 DBI 总线给用户使用。

dbi_init_mux 模块在被复位时，才会被触发，该模块复位需要和 PCIE IP 的复位一致。复位结束后，用户可以自由使用 ext_ltssm_enable 和 DBI 总线。

需要注意当用户通过 DBI 总线读写 PCIE 配置时，由于自恢复模块，中间存在部分延迟和打拍的现象。用户需要注意 DBI 的时序关系。

3.6.2 IP 及参数配置

在软件中用户需要勾选 PCIE IP 中，Export DAT file 选项，即可生成同名的.dat 文件，用于 rom 初始化。

PCIe Basic	Capability 1	Capability 2	ID and Class	BAR	Port Control
Device Type:	Legacy PCI Express Endpoint				
PCIe Generation:	Gen3				
Link Width:	X4				
Link Location:	Bank 83 and Bank 82				
Reference Clock Frequency:	100Mhz				
PCIe Application Mode:	Advanced				
File Exporting					
☒ Export DAT File					

图 3-22 生成初始化文件

Rom 的 GUI 生成界面如下:

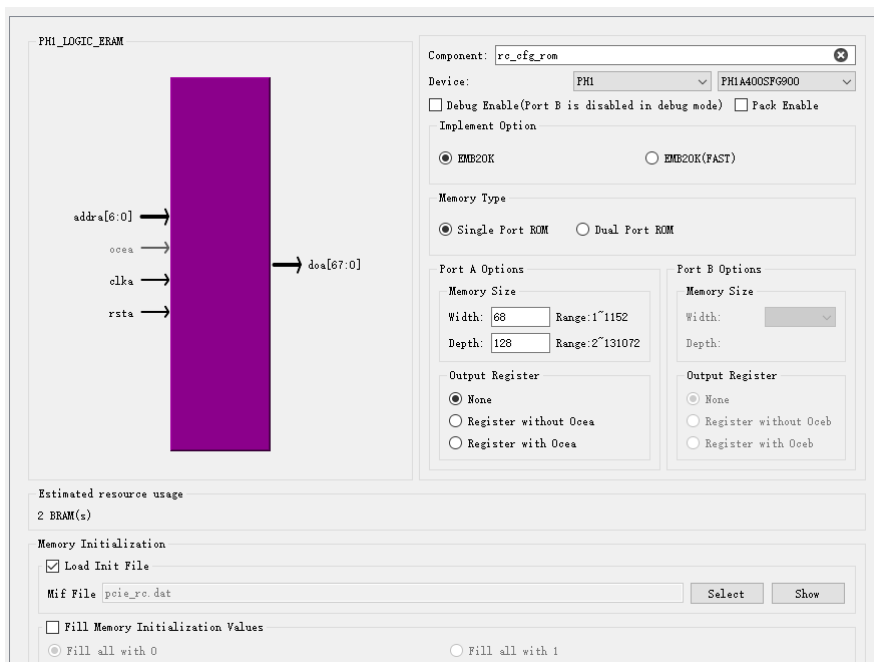


图 3-23 生成初始化 ROM

3.6.3 DBI 总线配置模块

PCIe IP 的配置不同,生成配置文件大小也各不相同,dbi_init_mux 模块中会自动读取 Rom 中的数据,用以适配 PCIe 配置文件。dbi_init_mux 模块是 fabric 逻辑,接口信号主要分为三类,即时钟复位信号,dbi 总线信号和 ltssm_enable 信号。

当用户需要复位 PCIe 时,dbi_init_mux 模块也需要同时被复位,复位解除后就会拉低 app_app_ltssm_enable 延迟链路训练,并且从 Rom 中读取 PCIe 配置信号,通过 dbi 总线配置。

配置完成后,dbi_init_mux 模块会将 PCIe IP 的 app_app_ltssm_enable 和 dbi 总线透传到 ext_* 信号上,供用户复位之后使用。

4 仿真

参考设计支持 Modelsim 仿真，在 sim 目录下包含 modelsim 子目录，进入该目录后，运行仿真脚本即可进行仿真。

4.1 运行仿真

将 Modelsim/Questasim 切换到 sim 目录后，在命令窗口输入：do sim.do。

软件即开始启动编译和运行仿真。仿真时长限制在 500us 左右，如果打印如下带有 OK 的内容，则表示成功。用户可自行添加信号波形观察。

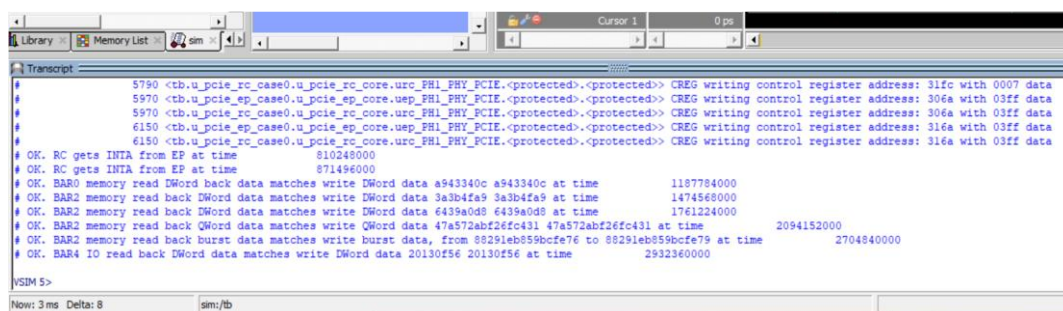


图 4-1 modelsim 仿真结果

需要注意，用户在进行仿真之前，需要修改工程相关的路径设定，在 sim.do 中，需要重新设定仿真库的路径。

4.2 仿真结构

参考设计中的仿真用例演示了：

- 配置读/写
- Memory 单 Dword 读/写
- Memory Qword 读/写
- Memory Burst（32/256 字节）读/写
- IO 读/写
- Legacy Int、MSI、MSIX 中断

Testbench 的顶层模块图如下：

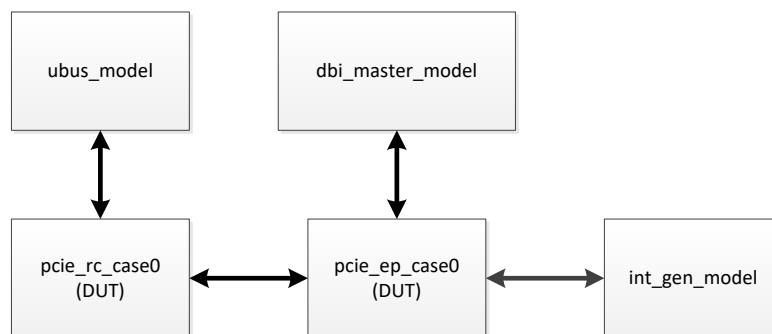


图 4-2 仿真结构图

其中的 ubus_master 例化 ubus_model 模拟了主机通过 uart 及 ubus 下发 Config/Memory/IO 的读写操作，已经封装到 ubus_task.v 里的 task 中。dbi_master_model 模块模拟了 fpga 内通过 DBI 接口直接配置 EP, 有关操作已经封装到 dbi_master_model.v 里的 task 中。

参考设计支持三种中断，但是只能同时支持一种中断，具体修改中断方式，见 3.3 章节。

RC 侧模拟主机的 UART 和 ubus 的具体模块图如下：

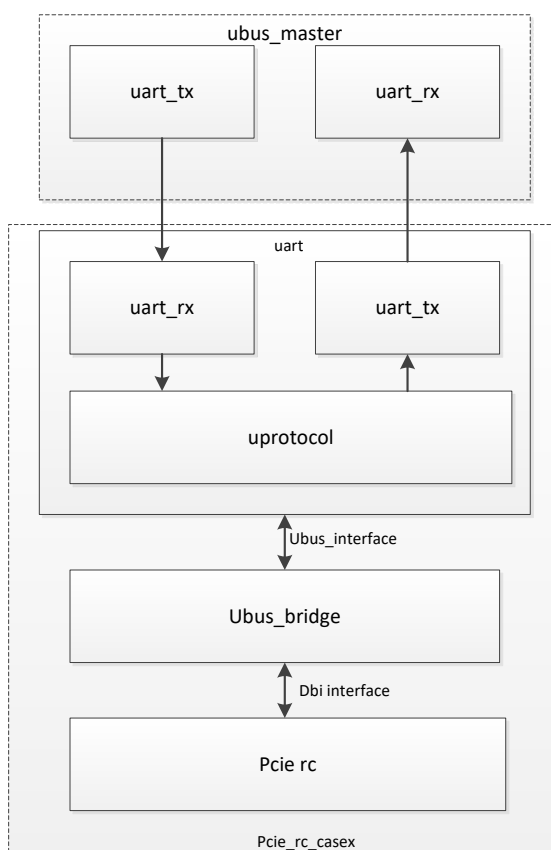


图 4-3 RC 侧仿真模块结构图

用户可以观察参考设计仿真波形图，结合代码理解信号或者模块的功能，例如当 RC 发送 Configuration write 0 TLP 报文时，快速仿真模式中，大致流程如下：

- 写 32'h0000_0002 到 32'hFFFF_FF01
- 写寄存器地址到 32'hFFFF_FF10
- 写寄存器数据到 32'hFFFF_FF11
- 写全 0 到 32'hFFFF_FF02
- 读 32'hFFFF_FF02, 检查 bit[1] 是否为 1。
- Bit[1] 为 1, 写全 0 到 32'hFFFF_FF01

RC Client0 总线发送 request 时序如下图:

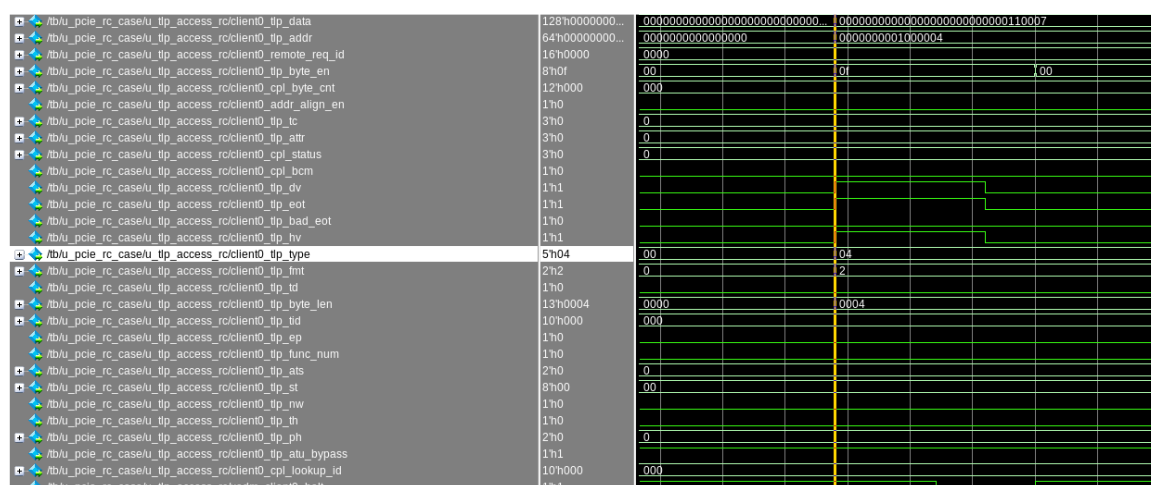


图 4-4 RC 侧仿真结构图

上图中, client0_tlp_hv/ client0_tlp_dv/ client0_tlp_eot 同时拉高, 直到 xadm_client0_halt 变低才拉低。client0_tlp_fmt 和 client0_tlp_type 编码分别为 0x2 和 0x04, 表示 CFGWR0。client0_tlp_byte_en 为 0x0F, client0_tlp_byte_len 为 0x0004, 写操作长度为 4byte。

收到响应时, RC Bypass 总线收到的 completion 时序如下图:



nbu_pcie_rc_case/u_tip_access_rcradm_bypass_data	128h000000...	00000000000000000000000000000000	00000000000000000000000000000000
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_dwen	4h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_dv	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_hv	1h1		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_eot	1h1		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_dlip_abort	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_tip_abort	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_ecrc_err	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_addr	64h00000000...	00000000000000000000000000000000	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_fmt	2h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_rc	3h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_atr	3h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_reqid	16h0000	0000	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_type	5h0a	00	0a
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_tag	10h000	000	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_func_num	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_td	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_poisoned	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_dw_len	10h000	000	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_rom_in_range	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_first_be	4h4	0	4
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_last_be	4h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_io_req_in_range	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_in_membar_range	3h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_cpl_last	1h1		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_cpl_status	3h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_st	8h00	00	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_cmpltr_id	16h0100	0000	0100
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_byte_cnt	12h004	000	004
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_ats	2h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_th	1h0		
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_ph	2h0	0	
nbu_pcie_rc_case/u_tip_access_rcradm_bypass_bcm	1h0		

图 4-5 RC 侧仿真模块结构图

上图中, radm_bypass_hv/radm_bypass_eot 同时拉高一个时钟周期。radm_bypass_fmt 和 radm_bypass_type 编码分别为 0x0 和 0x0a, 表示 CPL。radm_bypass_cpl_status 为 0, 表示成功完成。

5 编译与调试

5.1 配置说明

实现 RC 所在的 Demo 板将为 RC 和 EP 提供参考时钟。请注意检查。

对于 400K, RC 所在 Demo 板 R208/R209 必须焊上, 即由本地 PLL 产生参考时钟, RC 代码中已经包含了该部分, Demo 板 R208/R209 位置如下图:



图 5-1 RC 端 R208/R209 位置

EP 所在 Demo 板无需修改;

对于 90K, RC 所在 Demo 板 R48/R49 必须焊上, 即由本地 PLL 产生参考时钟, RC 代码中已经包含了该部分, Demo 板 R48/R49 位置如下图:



图 5-2 RC 端 R48/R49 位置

EP 所在 Demo 板无需修改;

上述两个电阻在电路板上的位置在 Demo 板正面 PCIe 金手指附近。

5.2 RC 连接 EP 测试过程

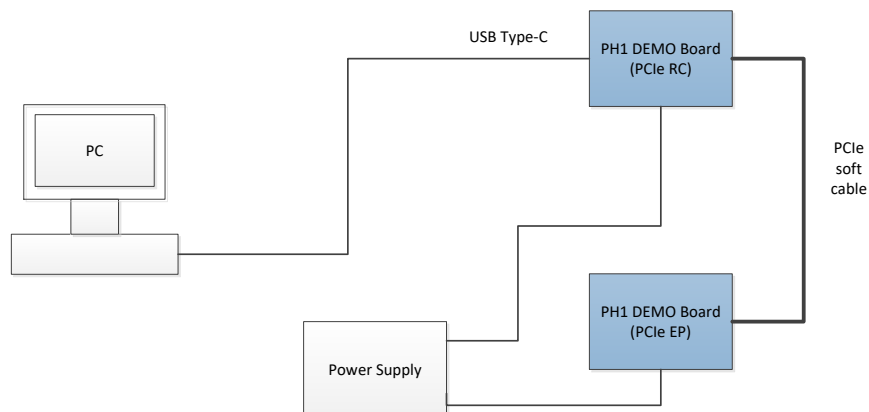


图 5-3 RC 连接 EP 拓扑图

RC-EP 对连拓扑如上图所示。图中的 RC 和 EP 分别是 2 块 PH1 系列 Demo Board，2 板互连用 PCIe 软排线。JTAG cable 用于板上 FPGA 的下载。USB Type-C 是用于连接 PC 和 RC，因为 Demo Board 上有 USB 转 UART 的芯片，USB Type-C 连接会被 PC 识别成 COM 口。PC 将通过 COM 口，发起各种寄存器读写操作，RC 中会将寄存器的操作转化为 PCIe 的 TLP 事务。

400K Demo Board 真实连接如以下 2 张图片：



图 5-4 RC 连接 EP 1



图 5-5 RC 连接 EP 2

其中，上面的 Demo 板被设置为 RC，下面的 Demo 板被设置为 EP，板卡与板卡之间用螺钉堆叠起来并固定住，防止触碰造成板卡损坏，两个板卡之间 PCIe 接口，采用 PCIe 延长线的方式，连接两个金手指。

在使用软排线进行对接时，注意 RC 和 EP 板卡 PCIe 的正确对接。需要将 RC 的 PCIe TX 信号对接到 EP 的 PCIe RX 信号，将 RC 的 PCIe RX 信号对接到 EP 的 PCIe TX 信号。

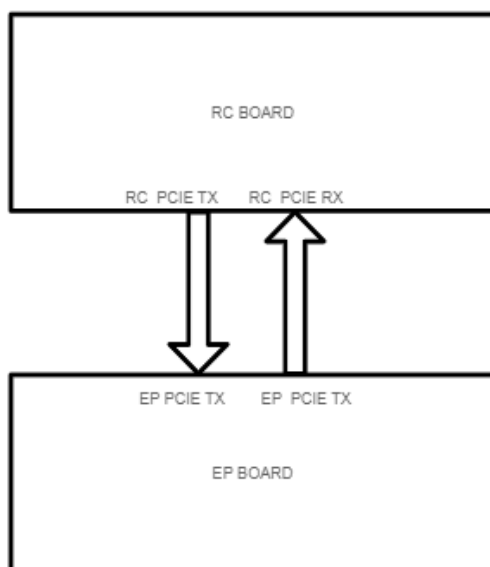


图 5-6 RC 连接 EP 拓扑图

参考设计使用两种不同的 PCIe 延长线。拓扑图如下：

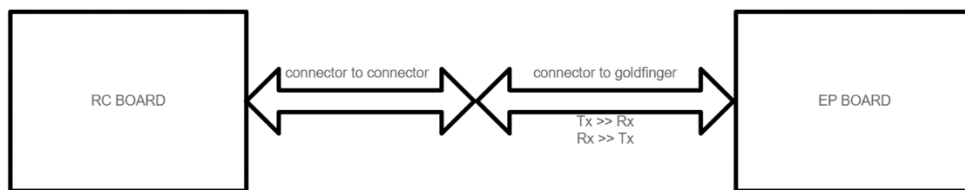


图 5-7 RC 连接 EP 拓扑图 2

先用母对母的延长线连接 RC 板卡和另一延长线的金手指部分，然后在将公转母的母端，连接到 EP。母对母的延长线只有延长信号的作用，无法将 PCIe RX 和 TX 信号对接，而公转母延长线可以将 RX 和 TX 对接。

5.2.1 TLP 测试过程

usb 插入板卡，会显示两个串口，使用端口号高的串口，板卡上电之后，用户在测试过程中，把 ubus_cmd.txt 里的测试命令拷贝粘贴到串口调试工具里，点击发送即可。

例如：对 EP 的配置寄存器 0x0000_0004 的 bit[2:0] 写 3' b111。将 ubus_cmd.txt 中对应的行复制到串口发送窗口中即可。

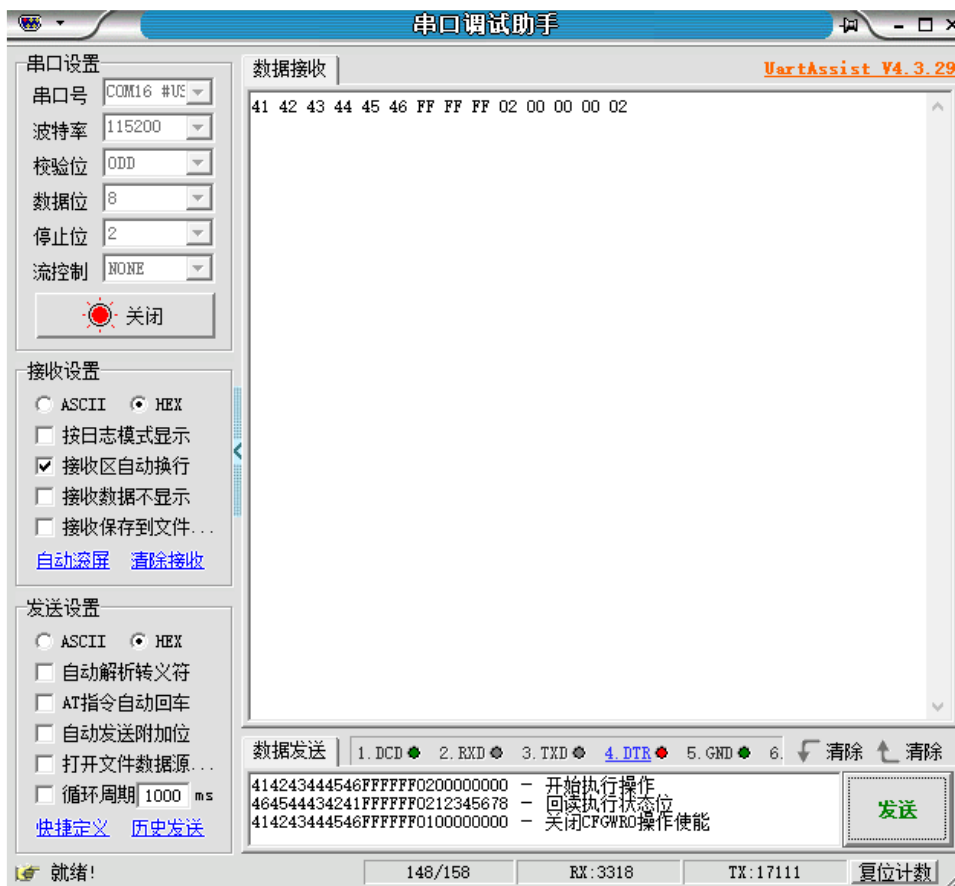


图 5-8 RC 端串口发送 EP 配置命令



例如需要对 EP 的配置寄存器 0x0000_0004 的 bit[2:0] 写 3'b111, 即使能 IO 和 MEM Space, 只需要按照以下格式通过串口发送即可:

```
414243444546FFFFFFF0100000002 – 打开CFGWR0操作使能
414243444546FFFFFFF1000000004 – 设置EP配置空间寄存器地址0x0000_0004
414243444546FFFFFFF1100110007 – 设置EP配置空间寄存器地址0x0000_0004的数据0x0011_0007
414243444546FFFFFFF0200000000 – 开始执行操作
464544434241FFFFFFF0212345678 – 回读执行状态位
414243444546FFFFFFF0100000000 – 关闭CFGWR0操作使能
```

图 5-9 RC 配置 EP 命令格式

ubus_cmd.txt 中包含了十几条命令, 几乎涵盖了 PCIe 常用的 TLP 报文的测试, 用户可按照文本中的提示, 进行测试验证。

5.2.2 中断测试过程

Legacy int 的测试, EP 端按下 Demo 板上绑定 local_int 的按键, 即可触发 Legacy int 中断, 每按下一次按键, 就会触发一次中断。

先读取中断状态寄存器:

```
464544434241FFFFFFF1412345678 读中断状态寄存器
```

COM 口返回的数据应该是:

```
414243444546FFFFFFF1400000001 中断状态寄存器是 1, 有 INTA 中断
```

清除中断状态寄存器:

```
414243444546FFFFFFF1400000001 清除中断状态寄存器
```

```
414243444546FFFFFFF1400000000 允许中断寄存器发生改变
```

MSI 测试同理。需要注意在测试中断时, 检查代码和 IP 是否已经支持了对应的中断。EP 端按下 Demo 板上绑定 local_int 的按键, 即可触发 MSI 中断, 每按下一次按键, 就会触发一次中断。

先读取中断状态寄存器:

```
464544434241FFFFFFF1512345678 读 MSI 中断寄存器
```

COM 口返回的数据应该是:

```
414243444546FFFFFFF1500000001 中断状态寄存器是 1, 有 MSI 中断
```

查看当前中断向量状态:

```
464544434241FFFFFFF16FFFFFFF 第一次触发按键返回中断向量 01, 依次 02, 04, ...40, 80
```

清除中断状态寄存器:



414243444546FFFFFF17FFFFFFF 清除 MSI 中断状态

414243444546FFFFFF1700000000 恢复 MSI 中断状态

MSIX 测试同 MSI 测试类似。需要注意在测试中断时，除检查代码和 IP 是否已经支持了对应的中断外，需通过串口设置 MSIX 中断向量和地址。EP 端按下 Demo 板上绑定 local_int 的按键，即可触发 MSI 中断，每按下一次按键，就会触发一次中断。ubus_cmd.txt 里的测试命令中设置了 4 个中断向量的地址和数据。进行中断测试时，将支持检测四种中断向量状态。

464544434241FFFFFF1512345678 读取 MSIX 中断状态

464544434241FFFFFF16FFFFFFF 查看 MSIX 中断向量

414243444546FFFFFF17FFFFFFF 清除 MSIX 中断状态

414243444546FFFFFF1700000000 恢复 MSIX 中断状态

5.2.3 动态切换

加载 90K, 400K 相应的 RC 和 EP 的 bit 文件之后，gen1_gen2 默认上电运行在 gen2x4 的模式，gen3 默认上电运行在 gen3x4 模式，其中 gen1_gen2 可进行速度和位宽的更改，gen3 不建议进行速度和位宽的切换。如果想改变位宽或者速率，需要通过 RC 端串口与 RC PCIe DBI 总线通信来实现。

动态切换速度，实现过程

切换之前，我们先读当前的线速寄存器，发 UBUS 命令：

4645444342410000008012345678 读取 PCIe IP 0x80 寄存器

COM 口返回的数据可能是如下 2 种：

41 42 43 44 45 46 00 00 00 80 B0 41 00 00 当前线速是 Gen1

41 42 43 44 45 46 00 00 00 80 B0 42 00 00 当前线速是 Gen2

如果需要切换到 Gen1，先发 UBUS 命令：

414243444546000000A001010001 设置 target link speed 为 Gen1，0xA0 寄存器

4142434445460000080C000302BE 设置 Directed Speed Change 为 1，0x80C 寄存器

4142434445460000080C000102BE 设置 Directed Speed Change 为 0，0x80C 寄存器

如果要切换到 Gen2，先发 UBUS 命令：

414243444546000000A001010002 设置 target link speed 为 Gen2，0xA0 寄存器

4142434445460000080C000302BE 设置 Directed Speed Change 为 1，0x80C 寄存器

4142434445460000080C000102BE 设置 Directed Speed Change 为 0，0x80C 寄存器



以上步骤完成后，线速就切换成功了。LED2 的闪烁频率将发生明显变化。如果想读切换后的线速寄存器，操作跟之前读线速寄存器一样。

动态切换速度，实现原理

动态切换速度实际上是由 RC 发起的链路重新协商训练，实现的过程都是由用户控制 PCIe IP DBI 总线，从而控制 PCIe 内部寄存器来实现的。主要可以分为以下四步：

第一步，在切换之前，可以通过 DBI 读取 0x80 寄存器，该寄存器 bit16:19 位为当前协商速度，0x1 表示 gen1，0x2 表示 gen2，以此类推。如果用户在切换之前，不需要知道当前速度和位宽，可跳过该步骤。

第二步，切换速度时，需要将所切换的速度，通过 DBI 写入到 0xa0 寄存器，0xa0 寄存器 bit[3:0] 为预设定的速度。

第三步，实现切换时，需要控制 0x80c 寄存器，该寄存器 bit17 位为 Directed Speed Change，置 1 表示控制控制器速度发生变化，需要重新进行协商。

第四步，完成切换后，同样控制 0x80c 寄存器，将 bit17 位 Directed Speed Change，由上一步写 1，改回 0，表示速度切换完成。

动态切换位宽，实现过程

切换之前，我们先读当前的线速寄存器，发 UBUS 命令：

4645444342410000008012345678 读取 PCIe IP 0x80 寄存器

COM 口返回的数据可能是如下 3 种：

41 42 43 44 45 46 00 00 00 80 B0 42 00 00 当前位宽是 x4

41 42 43 44 45 46 00 00 00 80 B0 22 00 00 当前位宽是 x2

41 42 43 44 45 46 00 00 00 80 B0 12 00 00 当前位宽是 x1

如果需要切换到 x4，发 UBUS 命令：

414243444546000008C0000000C4 设置位宽为 x4, 且 Directed link width Change 为 1, 0x8C0 寄存器

414243444546000008C004000004 设置位宽为 x4, 且 Directed link width Change 为 0, 0x8C0 寄存器

如果要切换到 x2，发 UBUS 命令：

414243444546000008C0000000C2 设置位宽为 x2, 且 Directed link width Change 为 1, 0x8C0 寄存器

414243444546000008C000000002 设置位宽为 x2, 且 Directed link width Change 为 0, 0x8C0



寄存器

如果要切换到 x1，发 UBUS 命令：

414243444546000008C0000000C1 设置位宽为 x1, 且 Directed link width Change 为 1, 0x8C0
寄存器

414243444546000008C000000001 设置位宽为 x1, 且 Directed link width Change 为 0, 0x8C0
寄存器

以上步骤完成后，位宽就切换成功了，重复读取 0x80 寄存器，查看位宽是否变化。

动态切换位宽，实现原理

动态切换位宽实际上是由 RC 发起的链路重新协商训练，实现的过程都是由用户控制 PCIe IP DBI 总线，从而控制 PCIe 内部寄存器来实现的。主要可以分为以下三步：

第一步，在切换之前，可以通过 DBI 读取 0x80 寄存器，该寄存器 bit23:20 位为当前协商位宽，0x1 表示 x1, 0x2 表示 x2，以此类推。如果用户在切换之前，不需要知道当前位宽，可跳过该步骤。

第二步，切换位宽时，需要将所切换的位宽，通过 DBI 写入到 0x8c0 寄存器，0x8c0 寄存器 bit[3:0] 为预设定的位宽，bit6 为 Directed link width Change，需要同时写入修改的位宽和设置 Directed link width Change 为 1。

第三步，完成切换后，同样控制 0x8c0 寄存器，将 bit6 位 Directed link width Change，由上一步写 1 改回 0。表示速度切换完成。

5.3 EP 连接 PC 测试过程

EP 的工程不仅可以连接 RC 进行测试，也可以连接 PC 进行测试，PC 可以读写 EP 端 BAR0/BAR2/BAR4。也可以检测中断的生成。目前 PC 端驱动只提供 Linux 版本。Windows 版本尚未支持。需要用户将 driver 目录拷贝到 linux 下。

5.3.1 TLP 测试过程

第一步，设备检测。PH1A400 AP103_V2.0 Demo 板上电后，先加载附件中对应的 EP 端位流文件，重启 PC 机，进入到 Linux 系统，使用 lspci 命令查看 PCIe 状态，找到对应 pcie bus 号，再查看详细的 PCIe 设备信息。

在加载相应位流信息，并重启 PC 机后，可通过 lspci 查看 pcie bus 号 01:00.0，并通过 lspci -s 01:00.0 -vvv 查看设备具体信息。



```
[root@localhost ~]# lspci -s 01:00.0 -vvv
01:00.0 RAM memory: Device 1edb:dcba
Control: I/O- Mem- BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr- Stepping- SERR+ FastB2B- DisINTx-
Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
Interrupt: pin A routed IRQ255
Region 0: Memory at a1200000 (32-bit, non-prefetchable) [disabled] [size=1M]
Region 2: Memory at a1100000 (32-bit, non-prefetchable) [disabled] [size=1M]
Region 4: I/O ports at 3000 [disabled] [size=256]
Capabilities: [40] Power Management version 3
Flags: PMEClk- DSI- D1+ D2- AuxCurrent=375mA PME(D0-,D1-,D2-,D3hot-,D3cold+)
Status: D0 NoSoftRst+ PME-Enable- DSel=0 DScale=0 PME-
Capabilities: [70] Express (v2) Legacy Endpoint, MSI 00
DevCap: MaxPayload 2048 bytes, PhantFunc 0, Latency L0s unlimited, L1 unlimited
ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset-
DevCtl: CorrErr- NonFatalErr- FatalErr- UnsupReq-
RlxdOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop-
MaxPayload 256 bytes, MaxReadReq 512 bytes
DevSta: CorrErr+ NonFatalErr- FatalErr- UnsupReq+ AuxPwr+ TransPend-
LnkCap: Port #0, Speed 8GT/s, Width x4, ASPM not supported
ClockPM- Surprise- LLActRep- BwNot- ASPMOptComp+
LnkCtl: ASPM Disabled; RCB 64 bytes, Disabled- CommClk-
ExtSynch- ClockPM- AutWidDis- BWInt- AutBWInt-
LnkSta: Speed 8GT/s (ok), Width x4 (ok)
TrErr- Train- SlotClk- DLActive- BWMgmt- ABWMgmt-
DevCap2: Completion Timeout: Not Supported, TimeoutDis+ NROPrPrP- LTR+
10BitTagComp+ 10BitTagReq- OBFF Not Supported, ExtFmt- EETLPPrefix-
EmergencyPowerReduction Not Supported, EmergencyPowerReductionInit-
FRS+
AtomicOpsCap: 32bit+ 64bit+ 128bitCAS-
DevCtl2: Completion Timeout: 50us to 50ms, TimeoutDis- LTR+ OBFF Disabled,
AtomicOpsCtl: ReqEn-
LnkCap2: Supported Link Speeds: 2.5-8GT/s, Crosslink- Retimer+ 2Retimers+ DRS+
LnkCtl2: Target Link Speed: 8GT/s, EnterCompliance- SpeedDis-
Transmit Margin: Normal Operating Range, EnterModifiedCompliance- ComplianceSOS-
Compliance De-emphasis: -6dB
LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete- EqualizationPhase1-
EqualizationPhase2- EqualizationPhase3- LinkEqualizationRequest-
Retimer- 2Retimers- CrosslinkRes: Upstream Port
```

图 5-10 lspci 查看 PCIe 状态

上图所示中，参考设计的连接状态为 Gen3x4，并使能了三个 BAR，分别为 BAR0，BAR2，BAR4，BAR0 和 BAR2 为 memory type，大小为 1MB，BAR4 为 IO type，大小为 256bytes。

第二步，安装 PCIe 驱动。driver 目录中存在三个子目录，这三个子目录都可以进行 BAR 的读写，但是中断只能支持对应的一种。进入到 int/anlogic_pcie_drv 目录中，首先输入 make 命令编译驱动，然后输入 insmod anlogic_pcie.ko 即可成功安装驱动。

第三步，测试读写 BAR。BAR 的读写测试，是通过用户代码实现的，进入到 int/anlogic_pcie_app 目录中，输入以下指令对 pci_bar.c 文件进行编译：

```
gcc pci_bar.c -o pci_bar
```

生成应用程序后，根据目录中 readme.txt 中的指令对 BAR 进行读写测试：

(1) 读写 memory BAR0

```
[root@localhost anlogic_pcie_app]# ./pci_bar 0 0x0 0x12345678
Pci bar0: address: 0x0 write data: 0x12345678.
character device 0 opened.
Write complete.
[root@localhost anlogic_pcie_app]# ./pci_bar 0 0x0
Pci bar0: address: 0x0 read.
character device 0 opened.
Read data: 0x12345678 complete.
[root@localhost anlogic_pcie_app]# ./pci_bar 0 0x4 0xaa553344
Pci bar0: address: 0x4 write data: 0xaa553344.
character device 0 opened.
Write complete.
[root@localhost anlogic_pcie_app]# ./pci_bar 0 0x4
Pci bar0: address: 0x4 read.
character device 0 opened.
Read data: 0xaa553344 complete.
```

图 5-11 读写 memory BAR0

(2) 读写 memory BAR2

```
[root@localhost anlogic_pcie_app]# ./pci_bar 2 0x0 0x12345678
Pci bar2: address: 0x0 write data: 0x12345678.
character device 2 opened.
Write complete.
[root@localhost anlogic_pcie_app]# ./pci_bar 2 0x0
Pci bar2: address: 0x0 read.
character device 2 opened.
Read data: 0x12345678 complete.
[root@localhost anlogic_pcie_app]# ./pci_bar 2 0x4 0xaa5533443
Pci bar2: address: 0x4 write data: 0xaa5533443.
character device 2 opened.
Write complete.
[root@localhost anlogic_pcie_app]# ./pci_bar 2 0x4
Pci bar2: address: 0x4 read.
character device 2 opened.
Read data: 0xaa5533443 complete.
```

图 5-12 读写 memory BAR2

(3) 读写 IO BAR4

```
[root@localhost anlogic_pcie_app]# ./pci_bar 4 0x0 0xaa5599aa
Pci bar4: address: 0x0 write data: 0xaa5599aa.
character device 4 opened.
Write complete.
[root@localhost anlogic_pcie_app]# ./pci_bar 4 0x0
Pci bar4: address: 0x0 read.
character device 4 opened.
```

图 5-13 读写 IO BAR4

上图中可以看到，写入数据和读出数据一致，说明读写 BAR 成功。

5.3.2 中断测试过程

Legacy int 的测试，驱动只能使用 driver/ Legacy int 中的驱动，按下 Demo 板上按键，即可触发 Legacy int 中断，每按下一次按键，就会触发一次中断。使用 dmesg 命令查看 PC 是否收到中断。需要注意在测试中断时，检查代码和 IP 是否已经支持了对应的中断。

```
[ 139.383280] allocate cdev region OK
[ 139.383280] cdev region MAJOR: 509, MINOR:0
[ 139.383281] config_kobject before
[ 139.383281] config_kobject after
[ 139.383281] cdev_ctrl_init before
[ 139.383282] xcdev 0x00000000175bbd70, 509:1, (null), type 0x1.
[ 139.383284] create_sys_device before
[ 139.383405] xpdev_create_interfaces ctrl_cdev OK
[ 139.383406] xpdev_create_interfaces user bar:-1
[ 182.763018] cmd 0x40107804, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 189.186984] cmd 0x80107805, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 198.586847] cmd 0x40107804, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 206.378845] cmd 0x80107805, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 228.978945] cmd 0x40107804, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 233.267035] cmd 0x80107805, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 240.339360] cmd 0x40107804, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 244.234966] cmd 0x80107805, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 257.955322] cmd 0x40107804, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 261.387136] cmd 0x80107805, xdev 0x000000006c00bf58, pdev 0x000000006e418941.
[ 390.163133] Interrupt Handler Start ..
[ 390.163140] (irq=16, dev 0x000000006c00bf58) <<<< ISR.
```

图 5-14 dmesg 查看 Legacy int 中断

MSI 的测试，驱动只能使用 driver/MSI 中的驱动，按下 Demo 板上按键，即可触发 MSI 中断，使用 dmesg 命令查看 PC 是否收到中断。需要注意在测试中断时，检查代码和 IP 是否已经支持了对应的中断。

```
[ 504.311627] config_kobject after
[ 504.311628] cdev_ctrl_init before
[ 504.311631] xcdev 0x00000000e25a62aa, 509:1, (null), type 0x1.
[ 504.311638] create_sys_device before
[ 504.311781] xpdev_create_interfaces ctrl_cdev OK
[ 504.311784] xpdev_create_interfaces user bar:-1
[ 589.363629] cmd 0x40107804, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 608.003792] cmd 0x80107805, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 624.715792] cmd 0x40107804, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 632.659634] cmd 0x80107805, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 690.595442] cmd 0x40107804, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 705.667610] cmd 0x80107805, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 718.011501] cmd 0x40107804, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 722.075545] cmd 0x80107805, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 792.891651] cmd 0x40107804, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 806.899696] cmd 0x80107805, xdev 0x000000007aeb4008, pdev 0x00000000a891cf16.
[ 1157.365962] Interrupt Handler Start ..
[ 1157.365969] (irq=143, dev 0x000000007aeb4008) <<<< ISR.
[ 1157.365981] (irq=143, dev 0x000000007aeb4008) <<<< ISR.
```

图 5-15 dmesg 查看 msi 中断

MSIX 的测试，驱动只能使用 driver/MSIX 中的驱动，按下 Demo 板上按键，即可触发 MSIX 中断，使用 dmesg 命令查看 PC 是否收到中断。需要注意在测试中断时，检查代码和 IP 是否已经支持了对应的中断。

```
[ 124.620119] create_xcdev zzzz
[ 124.620122] allocate cdev region OK
[ 124.620124] cdev region MAJOR: 244, MINOR:0
[ 124.620126] config_kobject before
[ 124.620129] config_kobject after
[ 124.620131] cdev_ctrl_init before
[ 124.620136] xcdev 0xffff880473b23030, 244:1, (null), type 0x1.
[ 124.620138] create_sys_device before
[ 124.620984] xpdev_create_interfaces ctrl_cdev OK
[ 124.620992] xpdev_create_interfaces user bar:-1
[ 140.485173] (irq=140) <<<< INTERRUPT service ROUTINE
```

图 5-16 dmesg 查看 MSIX 中断



6 参考文献

1. 《UG916_安路科技 AP103 开发板使用说明》
2. 《UG913_安路科技 PH1A 系列 FPGA PCIE 用户手册》



7 版本信息

日期	版本	修订记录
2021/8/15	v1.0	修改至可发布版本
2021/12/3	v1.1	1. si5345 模块时钟生成改为 auxclk 2. rc 端串口数据和地址改为正序 3. rc 端串口增加奇偶校验 4. rc 端串口模块时钟改为 auxclk，增加跨时钟域功能 5. rc 端无需按键切换 DBI 与 tlp 通信
2022/6/1	v1.2	1. 取消 clock aligner 使能 2. PCIE IP 统一使用 app_power_on_rst 复位 3. 增加 pll，帮助时序收敛
2023/2/14	v1.3	1. 修改代码支持 gen3 2. 增加 90K 支持 3. 修改 3.6 节自恢复模块



版权所有©2023 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其他方式授予任何知识产权许可；本文档仅为向用户提供使用器件的参考，协助用户正确地使用安路科技产品之用，其著作权归安路科技所有；本文档所展示的任何产品信息均不构成安路科技对所涉产品或服务作出任何明示或默示的声明或保证。

安路科技将不定期地对本文档进行更新、修订。用户如需获取最新版本的文档，可通过安路科技的官方网站（网址为：<https://www.anlogic.com>）自行查询下载，也可联系安路科技的销售人员咨询获取。